

Lost in Transmission: When and Why LLMs Fail to Reason Globally

Tobias Schnabel*
Microsoft Research

Kiran Tomlinson*
Microsoft Research

Adith Swaminathan
Netflix

Jennifer Neville
Microsoft Research

Abstract

Despite their many successes, transformer-based large language models (LLMs) continue to struggle with tasks that require complex reasoning over large parts of their input. We argue that these failures arise due to capacity limits on the accurate flow of information within LLMs. To formalize this issue, we introduce the bounded attention prefix oracle (BAPO) model, a new computational framework that models bandwidth constraints on attention heads, the mechanism for internal communication in LLMs. We show that several important reasoning problems like graph reachability require high communication bandwidth for BAPOs to solve; we call these problems BAPO-hard. Our experiments corroborate our theoretical predictions: GPT-4o, Claude, and Gemini succeed on BAPO-easy tasks and fail even on relatively small BAPO-hard tasks. BAPOs also reveal another benefit of chain of thought (CoT): we prove that breaking down a task using CoT can turn any BAPO-hard problem into a BAPO-easy one. Our results offer principled explanations for key LLM failures and suggest directions for architectures and inference methods that mitigate bandwidth limits.

1 Introduction

Despite the empirical successes of transformer-based large language models (LLMs), they exhibit persistent failures on *global problems* that require integrating information across the entire input, such as chaining syllogisms [1], function composition [10], and formal language recognition [5]. Our core hypothesis is that these failures arise due to an inability of LLMs to accurately communicate information across *residual streams* [13], the sequences of transformer blocks corresponding to each input token. For an LLM to solve a problem, information about early tokens must be transmitted through attention into the last token’s residual stream. While some problems, such as needle-in-a-haystack tasks, do not require much information to cross residual streams, we show that global problems with complex inter-token dependencies require substantial communication. We conjecture that LLMs fail on such problems due to a limit on the amount of information they can accurately transmit between residual streams, which we refer to as their *effective bandwidth*. This is supported by prior work on capacity bounds on attention [12] and sparse attention [9, 4, 32, 41]. Causal attention exacerbates communication issues by forcing preprocessing in early streams to be independent of later tokens, increasing the representation size of problems. Figure 1 illustrates the issue of information flow and previews our empirical results.

To formally analyze these issues, we propose the *bounded attention prefix oracle* (BAPO) model—a new computational model designed to capture the communication constraints and causal attention of transformer-based LLMs. BAPOs capture how much information must be communicated between residual streams to solve a problem, abstracting away other details of the transformer architecture. When producing an output, there are two ways in which transformers can integrate information about the last token with information from earlier tokens (i.e., from a prefix of the input): they can attend to

*Equal contribution.

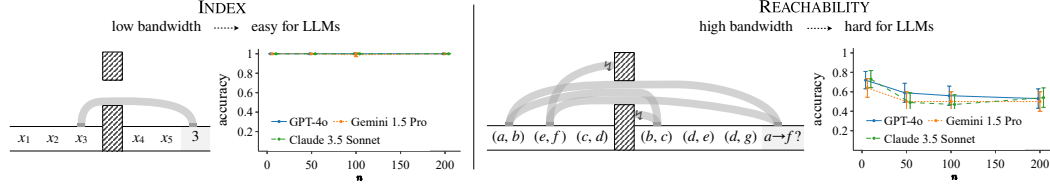


Figure 1: We conjecture that LLMs have a limit on their effective bandwidth, which we illustrate above by constrained information flow across one particular prefix–suffix split of the input. The BAPO model quantifies the communication bandwidth needed for transformers with causal attention to solve a problem. INDEX requires low communication bandwidth and LLMs solve it without issue; REACHABILITY requires high bandwidth and LLMs struggle with it.

precomputed values from higher layers of prefix streams, or they can attend to the raw prefix token values (or both; see Figure 2a). The BAPO model captures these two types of information flow and quantifies limits on the amount of information transmitted. In contrast with existing theoretical work on the expressivity of the transformer model class [40, 23, 15, 36, 33, 6], we seek to characterize the (in)ability of LLMs to solve global reasoning problems in practice. Strikingly, our experiments suggest that the effective bandwidth of modern LLMs is a small constant, so we call problems *BAPO-hard* if they cannot be solved by a constant-bandwidth BAPO and *BAPO-easy* if they can.

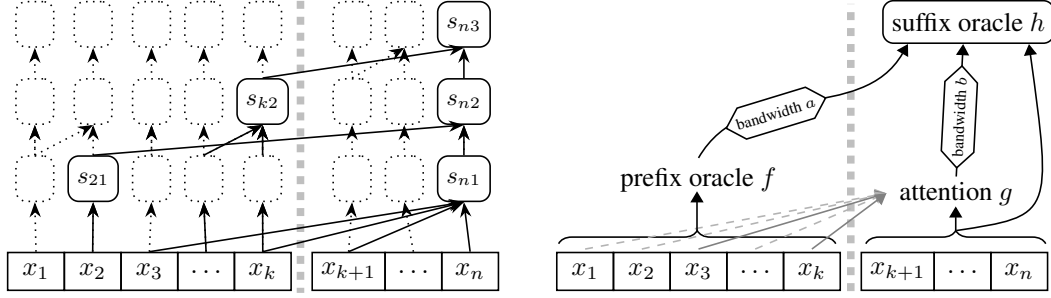
Theoretical contributions. We begin by highlighting the power of attention, which makes several hard communication problems (EQUALITY, DISJOINTNESS, and INDEX) BAPO-easy. In contrast, we show that a variety of important global problems are BAPO-hard, thus posing a challenge to LLMs with imprecise internal communication. In particular, we prove BAPO-hardness for REACHABILITY, MAJORITY, MATCH 3_n [33], UNIQUE, and SETDIFF, with lower bounds on the communication bandwidth required for each problem. On the positive side, we show that chain of thought (CoT) allows us to break down any BAPO-hard problem into a sequence of BAPO-easy steps, suggesting one mechanism for the success of CoT [24, 15, 29]. Specifically, we show that CoT renders constant-bandwidth BAPOs Turing-complete. This enables them to solve any decidable problem given a enough output tokens (although this number might be impractically large).

Empirical contributions. Our experimental results confirm the predictive power of the BAPO model: GPT-4o, Claude, and Gemini systematically fail to solve even relatively small instances of BAPO-hard problems, while performing well on BAPO-easy tasks across instance size (as previewed in Figure 1). We also demonstrate how real-world LLM tasks such as aggregating reviews and variable-tracking in code contain BAPO-hard components, namely MAJORITY and REACHABILITY, and thus present a challenge for LLMs. This illustrates the significance of our theoretical work in practice. Supporting our CoT result, the reasoning models o3 and Gemini 2.5 Flash perform very well even on BAPO-hard problems, albeit with a very large number of reasoning tokens. Our code is available at <https://github.com/microsoft/bapo>.

Implications. Identifying BAPO-hard and BAPO-easy subtasks enables practitioners to anticipate LLM limitations and proactively employ mitigation techniques like inference-time scaling, hybrid architectures, or tool-calling. A key feature of the BAPO model is that it abstracts away the low-level details of transformers and instead focuses on how much information must flow to solve a problem, yielding a characterization that can be applied more broadly and intuitively. Our work also shows how chain-of-thought reasoning can alleviate the communication needs of problems by breaking a problem down into steps requiring only a small amount of information flow, suggesting low bandwidth requirement as an additional objective in learning from reasoning chains. Ultimately, BAPOs offer an explanatory foundation for observed LLM failures on global reasoning problems and can unlock principled innovations to overcome these limitations.

2 A communication model of LLMs

The goal of our model is to represent the information processing flow within transformers while abstracting away lower level details. At a high level, recall that a transformer with causal attention



(a) A three-layer transformer with input tokens $x_1, \dots, x_n$. Node s_{ij} is the transformer block at position i and layer j . Information from the prefix needed for the solution output at s_{n3} is gathered by attending to either (i) past tokens (here, x_3 and x_k) or (ii) intermediate outputs (here, s_{21} and s_{k2}). The attention to prefix tokens is captured by the attention g of a BAPO, while the prefix oracle f of a BAPO models all other transmitted information from the prefix.

(b) An (a, b) -BAPO computes its result from the output of the prefix oracle f (limited to a bits), attention function g (limited to b tokens), and suffix tokens $k + 1, \dots, n$. The attention function can choose which tokens to attend to as a function of the suffix, but the decision to attend to each prefix token is independent from other prefix tokens. An arbitrary subset G of size b is received by h if g attends to more than b tokens. Every component has access to token indices.

Figure 2: A simplified view of a transformer and our bounded attention prefix oracle (BAPO) model.

makes next-token predictions at every token position in parallel. Additionally, no residual stream knows how far from the end of the input it is or what tokens come later in the input, due to causal attention. As such, if a transformer needs to solve a problem and we split its input into a prefix and suffix, any preprocessing done in the prefix streams should be useful no matter the suffix tokens. Moreover, information about the prefix must be communicated to the suffix streams, as they need to make next-token predictions that may depend on prefix tokens. This communication must occur across *every* possible prefix–suffix split, but for simplicity, we will model and analyze an arbitrary (usually worst case) split.

To make this more concrete, Figure 2a visualizes the computation of the next token x_{n+1} inside a transformer, with nodes contributing to the prediction of x_{n+1} highlighted (other attention weights are 0). A particular prefix–suffix split is shown with a dashed gray line, dividing the input tokens and their residual streams (i.e., the columns above each token). The output stream in the suffix must have all relevant information about the prefix tokens to solve a problem that depends on the whole input. That is, all needed information about the prefix tokens must cross the dashed gray line. This information can either come from (i) attending to input tokens directly (arrows crossing the split into s_{n1}) or (ii) attending to intermediate outputs from the prefix streams (arrows crossing the split into s_{n2} and s_{n3}). As we have emphasized, any intermediate outputs from prefix layers must be usable for all possible suffixes, since prefix streams cannot depend on future tokens due to causal attention.

Our central hypothesis is that LLMs have a limited ability to exactly communicate a large number of tokens or a large intermediate result across residual streams, thus causing failures on problems that require high information flow. We call this the *effective bandwidth* of an LLM. This hypothesis is informed by prior work that has derived capacity limits on attention heads [12], shown that problems requiring reasoning over many tokens pose a challenge [1], and proved that individual token impacts on attention tend to zero as the input length grows [18].

BAPOs. Our model isolates the issue of limited effective bandwidth as it plays a role in LLMs with causal attention. For simplicity, we consider problems with single-token solutions, which includes all decision problems. More formally, we consider tasks where an LLM is prompted with a fixed problem description $\mathcal{P} \in \Sigma^*$ concatenated with an input $x_1 \dots x_n \in \Sigma^*$, where Σ is the token vocabulary. The goal is to produce a solution $y \in \Sigma$, which we represent with the function $p : \Sigma^* \rightarrow \Sigma$ with $p(x_1 \dots x_n) = y$. We begin with an intuitive overview of the model.

Informal Definition 1. A *bounded attention prefix oracle* (BAPO; see Figure 2b) must solve a problem given an input split arbitrarily into a prefix and a suffix. A BAPO computes the solution given the suffix, a bits output by a *prefix oracle* f that accesses only the prefix, and b prefix tokens selected individually by a binary attention function g , with full positional information. The prefix

oracle f models the intermediate processing in prefix residual streams, which has no access to the suffix due to causal attention. The limits on the output size of f and on the number of tokens g may attend to are the key bandwidth constraints of the model, capturing limited attention head capacity.

Given this intuition, we provide the formal definition of BAPOs (using $\mathbb{N} = \mathbb{Z}_{>0}$), which makes explicit how BAPOs account for positional encodings and what happens if g tries to attend to too many tokens (intuitively, the BAPO must work given any set of b tokens to which g attends).

Definition 1. An (a, b) -BAPO (*bounded attention prefix oracle*) is defined by a *prefix oracle* $f : \Sigma^* \rightarrow \{0, 1\}^a$, an *attention function* $g : \Sigma^* \times \mathbb{N} \times \Sigma \times \mathbb{N} \rightarrow \{0, 1\}$, and a *suffix oracle* $h : \{0, 1\}^a \times \bigcup_{i=0}^b (\Sigma \times \mathbb{N})^i \times \Sigma^* \times \mathbb{N} \rightarrow \Sigma$. An (a, b) -BAPO *solves* a computational problem $p : \Sigma^* \rightarrow \Sigma$ if $h(f(x_1 \dots x_k), G, x_{k+1} \dots x_n, k) = p(x_1 \dots x_n)$ for all $k < n$ and all $G \subseteq \mathbb{G} = \{(x_i, i) : 1 \leq i \leq k, g(x_{k+1} \dots x_n, k, x_i, i) = 1\}$ with $|G| = \min\{b, |\mathbb{G}|\}$.

We call a the *prefix bandwidth* (measured in bits) and b the *attention bandwidth* (measured in tokens) of the BAPO. We call a problem *BAPO-easy* if it can be solved by a BAPO with constant bandwidths (w.r.t. n) and *BAPO-hard* otherwise. If a problem requires bandwidths that scale with $|\Sigma|$, we say it is *BAPO- Σ -hard*. Note that any problem with n -token inputs can be solved by a $(n \lceil \log_2 |\Sigma| \rceil, 0)$ -BAPO or by a $(0, n)$ -BAPO as the prefix oracle can forward the entire prefix or the attention function can attend to the entire prefix; we call this the trivial upper bound on BAPO complexity.

Assumptions. We briefly discuss some trade-offs in the underlying assumptions for BAPOs. On the generous side, we assume that the prefix streams and suffix streams have unbounded computational power. However, this fact is tempered by the fact that a BAPO must work for all possible prefix-suffix splits. Regarding the attention function, our model is pairwise in the sense that g can only look at a single prefix token x_i at a time, as in real transformers. BAPOs differ though in that g can base its attention decisions on the entire suffix. However, transformers can compensate for this by communicating information between the suffix streams across multiple layers. Our model also assumes perfect positional encoding, whereas this is a point of failure in real transformers [11]. Finally, the attention g can only operate on the token layer, whereas in transformers, attention can also attend to outputs of subsequent layers. However, this is offset by the ability of the suffix oracle in our model to perform arbitrary computation on the attended tokens.

3 BAPO theory

We prove that a variety of important global reasoning problems like graph reachability are BAPO-hard and therefore pose a challenge for LLMs under our effective bandwidth hypothesis. Table 1 summarizes our hardness and tractability results. Strikingly, we also prove that chain of thought enables constant-bandwidth BAPOs to solve all decidable problems, suggesting a reduction in required bandwidth as another mechanism for the empirical success of chain of thought.

3.1 One attention token is all you need to solve hard communication problems

Before turning to problems that BAPOs fail to solve, we first establish a separation from the standard one-way communication model [30] upon which BAPOs are based. First, BAPOs are at least as powerful: any problem solvable with $a(n)$ bits of one-way communication on n -bit inputs is also solvable by a $(a(n), 0)$ -BAPO by having the prefix oracle implement the communication protocol. However, adding even just a little attention makes BAPOs strictly more powerful than pure one-way communication. Strikingly, even $(1, 1)$ -BAPOs can solve DISJOINTNESS, EQUALITY, and INDEX, which are maximally hard problems for one-way communication requiring n bits of communication [30] (see Section A for full definitions). Low BAPO complexity suggests that LLMs should be able to solve these problems well, which is corroborated by our empirical results in Section 4. These problems are also known to be efficiently expressible by transformers [6]. Note also that INDEX is conceptually very similar to the common needle-in-a-haystack benchmark tasks.

Theorem 1. DISJOINTNESS and EQUALITY have $(1, 1)$ -BAPOs and INDEX has a $(0, 1)$ -BAPO.

Proof sketch. Access to token indices allows the attention function to attend to bits that would be counterexamples to EQUALITY or DISJOINTNESS, while the bit output by the prefix oracle is needed

Table 1: Overview of our BAPO upper and lower bounds in terms of (prefix bandwidth, attention bandwidth). n : input length, m : number of edges, c : any integer ≥ 3 , ϵ : arbitrary constant in $(0, 1)$, $b(n)$: any $o(n)$ function. Trivial upper bounds: $(n \lceil \log_2 |\Sigma| \rceil, 0)$ or $(0, n)$. Adding chain of thought (CoT) brings the upper bound down to $(2, 3)$ for all decidable problems, but may require a large number of CoT steps.

	Problem	Lower bound	Upper bound
BAPO-easy	INDEX (Thm. 1)		$(0, 1)$
	EQUALITY (Thm. 1)		$(1, 1)$
	DISJOINTNESS (Thm. 1)		$(1, 1)$
	MATCH 2_n (Thm. 4)		$(0, 1)$
BAPO-hard	REACHABILITY (Thm. 2)	$(o(m^{1/c} \log m), o(m^{1-2/c}))$	trivial
	MAJORITY (Thm. 3)	$(o(\log n), o(n^{1-\epsilon}))$	$(\lceil \log_2 n \rceil, 0)$
	MATCH 3_n (Thm. 4)	$(o(n/b(n)), b(n))$	trivial
BAPO- Σ -hard	UNIQUE (Thm. 5)	$(o(\Sigma /b(\Sigma)), b(\Sigma))$	$(2 \Sigma , 0)$
	SETDIFF (Thm. 6)	$(o(\Sigma /b(\Sigma)), b(\Sigma))$	$(\Sigma , 0)$

when the prefix contains all of x and some of y . INDEX is trivial thanks to the attention function, which can pick out the indexed token. See Section B for the full proof. $\square$

3.2 BAPO complexity

We now show that the following problems are BAPO-hard (see Section A for formal definitions):

- REACHABILITY: given a directed graph G and two nodes s, t , check if G has an s - t path.
- MAJORITY: determine whether a bitstring has strictly more ones than zeros.
- MATCH 3_n : given input $x \in \mathbb{Z}_m^n$, determine whether there are some $i, j \in [n]$ such that $x_n + x_i + x_j \equiv 0 \pmod{m}$. Additionally, MATCH 2_n (which we show is BAPO-easy) is the problem of determining whether there is some $i \in [n]$ such that $x_n + x_i \equiv 0 \pmod{m}$. These are last-token versions of MATCH2 and MATCH3 [33].
- UNIQUE (hard w.r.t. $|\Sigma|$): output any token that appears exactly once in the input.
- SETDIFF (hard w.r.t. $|\Sigma|$): output a token that is in the first input string but not the second.

We begin our hardness results with the important REACHABILITY problem, which encompasses many natural reasoning tasks, e.g., checking whether a conclusion follows from a chain of implications. We show that limited-bandwidth BAPOs cannot solve REACHABILITY, and our lower bound smoothly trades off between the prefix and attention bandwidth requirements. We provide a high-level sketch of the proof strategy, which is common to all of our hardness proofs. All full proofs omitted from the paper can be found in Section B.

Theorem 2. *No $(o(m^{1/c} \log m), o(m^{1-2/c}))$ -BAPO can solve REACHABILITY in graphs with n nodes and $m = \Omega(n)$ edges for any integer constant $c \geq 3$.*

Proof sketch. Suppose for a contradiction that some limited-bandwidth BAPO (f, g, h) solves the problem. We construct a family of prefixes and suffixes such that: (1) the prefixes have substantial overlap with each other, (2) the attention function g pays attention to tokens that are common to all prefixes and thus cannot distinguish between them, (3) the number of prefixes is sufficiently large that the prefix oracle f is not one-to-one by the pigeonhole principle, and (4) given any two prefixes for which f collides, we can find a suffix that results in different solutions when concatenated to the colliding prefixes. This construction gives us a contradiction: two instances of the problem with opposite answers which the BAPO cannot distinguish between, as it sees the same suffix, the same output of f , and the same set of attended tokens (given some adversarial choice of G). $\square$

We suspect this bound is not tight; closing the gap between Theorem 2 and the trivial $(m \lceil \log_2 m \rceil, 0)$ - and $(0, m)$ -BAPOs is an interesting open problem.

Next, we consider a simple problem where our lower bound is tight, MAJORITY. According to the circuit-based analysis of transformer expressivity that shows they are in TC^0 [23], this problem should be trivial, as it is solved by a single majority gate. However, as we will see in our experiments, LLMs struggle with MAJORITY; here, we show that BAPOs require super-constant bandwidth to solve it and that even near-linear attention bandwidth is insufficient to improve the prefix bandwidth requirement. Our proof follows the same high-level strategy as with REACHABILITY.

Theorem 3. *No $(o(\log n), o(n^{1-\epsilon}))$ -BAPO can solve MAJORITY on length n inputs for any $0 < \epsilon < 1$. This prefix bandwidth is tight: there is a $(\lceil \log_2 n \rceil, 0)$ -BAPO solving MAJORITY.*

In Section B, we show that increasing the attention bandwidth even beyond near-linear eventually reduces the prefix bandwidth lower bound for MAJORITY.

Turning to the next problem, Sanford et al. [33] showed that 1-layer transformers can efficiently solve MATCH2, but need to scale polynomially to solve MATCH3. To keep the single-token output of our problems, we consider the contained subproblems of outputting the last item, which we call MATCH2_n and MATCH3_n . We show that limited-bandwidth BAPOs can solve MATCH2_n but not MATCH3_n , forming a parallel with the results of Sanford et al. [33]. Our proof again follows the same high-level strategy as before, and as with REACHABILITY, our lower bound smoothly trades off between the two bandwidths.

Theorem 4. *For any $b(n) = o(n)$ with $b(n) \geq 1$, no $(o(n/b(n)), b(n))$ -BAPO can solve MATCH3_n over $\mathbb{Z}_{n_2}^n$. In contrast, there is a $(0, 1)$ -BAPO for MATCH2_n .*

Finally, we show two problems are BAPO- Σ -hard, starting with UNIQUE, the problem of finding an item that appears exactly once in a sequence. Here, the difficulty of the problem is parametrized by the size of the token vocabulary Σ rather than the length of the input n . A very similar approach applies to SETDIFF, for which we find the same bound.

Theorem 5. *Let $k = |\Sigma|$. For any $b(k) = o(k)$ with $b(k) \geq 1$, no $(o(k/b(k)), b(k))$ -BAPO can solve UNIQUE. This is tight for $b(k) = O(1)$, as there is a $(2k, 0)$ -BAPO solving UNIQUE.*

Theorem 6. *Let $k = |\Sigma|$. For any $b(k) = o(k)$ with $b(k) \geq 1$, no $(o(k/b(k)), b(k))$ -BAPO can solve SETDIFF. This is tight for $b(k) = O(1)$, as there is a $(k, 0)$ -BAPO solving SETDIFF.*

3.3 BAPOs with chain of thought

We now show that any (decidable) BAPO-hard problem can be broken down into a sequence of BAPO-easy steps in the spirit of *chain of thought* (CoT) [43]. Previous work has shown how CoT makes transformers Turing-complete [24]. We show that CoT has an even stronger benefit: adding CoT to BAPOs lets them solve all decidable problems *with constant bandwidth*, indicating that even LLMs with constant effective bandwidth are Turing-complete with CoT.

To formalize CoT in the BAPO setting, we repeatedly apply a fixed BAPO to the input concatenated with the BAPO’s previous output tokens, just as in autoregressive decoding in LLMs.

Definition 2. An (a, b) -bounded attention prefix oracle with chain of thought (BAPO-CoT) that solves a computational problem $p : \Sigma^* \rightarrow \Sigma$ is an (a, b) -BAPO over the token set $\Gamma \supseteq \Sigma \cup \{\square\}$ that solves some computational problem $p' : \Gamma^* \rightarrow \Gamma$ such that for all inputs $x \in \Sigma^*$ to p , there exists some sequence of strings $s_1, \dots, s_m \in \Gamma^*$ with the following properties. (1) $s_1 = x$ (the BAPO-CoT starts with x as its input), (2) $s_{i+1} = s_i p'(s_i)$ for all $i = 1, \dots, m-1$ (at each step, it produces some chain-of-thought token), (3) $p'(s_i) = \square$ if and only if $i = m-1$ (at the last step only, it outputs the halt token), and (4) $p'(s_{m-2}) = p(x)$ (it solves the problem before halting).

We say that a BAPO *decides* a language L if it solves $p(x) = 1[x \in L]$. Let $L^{\leq n} = \{x \in L : |x| \leq n\}$. We show that very low-bandwidth BAPO-CoTs exist for any decidable problem. The core idea is that simulating a single Turing machine step is a low-bandwidth problem.

Theorem 7. *Let L be a language decided by a Turing machine with $s(n)$ space and input alphabet $\Sigma = \{0, 1\}$. For any n , there exists a $(2, 3)$ -BAPO-CoT that decides $L^{\leq n}$.*

Proof sketch. The BAPO-CoT simulates a Turing machine M by writing out the contents of the tape and the state at every step of M ’s execution. The attention function attends to the prefix to retrieve the current state and the prefix oracle passes along the bit under the tape head (if they are not in the suffix), allowing the suffix oracle to simulate a step of M . See Section B for details. $\square$

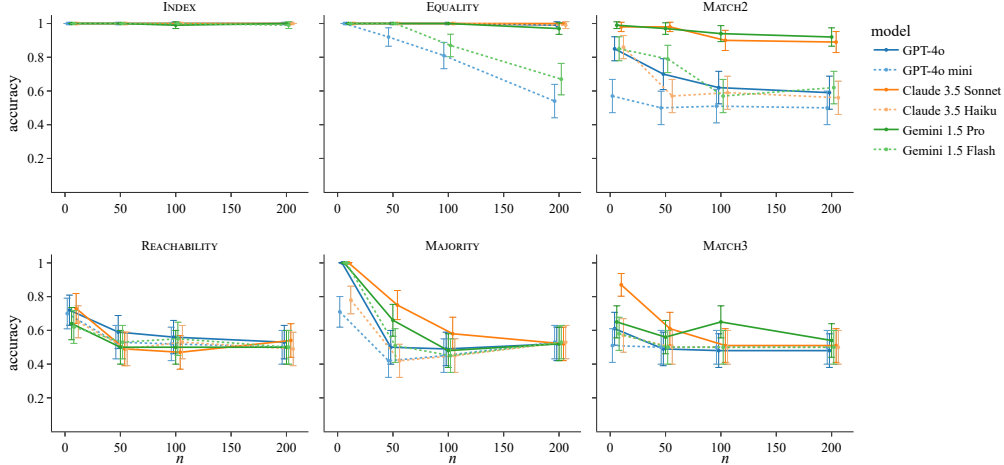


Figure 3: BAPO-hard problems (bottom row) show much larger drops in accuracy compared to BAPO-easy problems (top row). Not even large LLMs can solve BAPO-hard problems at length 200 with an accuracy above random guessing.

4 BAPO complexity predicts empirical LLM failures

Our experiments test whether BAPO complexity predicts LLM failures. Supporting our effective bandwidth hypothesis, we find that LLMs across model families consistently struggle with BAPO-hard problems and usually succeed on BAPO-easy problems. We choose three big model families (GPT [26], Gemini [16] and Claude [3]) and focus first on model versions without (latent) reasoning chains to align with the single-token-output BAPO model.

Except for INDEX and the BAPO- Σ -hard problems UNIQUE and SETDIFF, all problems have yes/no answers (so random guessing achieves 50% accuracy). We designed problem instances such that there would be no obvious shortcut or heuristic, pushing models to fully consider each problem. LLMs are fed with inputs of various lengths n , where n corresponds to the parameter specified in each problem’s definition. For all problems, we generate 100 i.i.d. instances and report average accuracy along with the 95% t -test confidence interval. All data generating distributions, prompts, and model details are available in Section C.1 and our code is available at <https://github.com/microsoft/bapo>.

4.1 BAPO hardness aligns well with LLM failures

Figure 3 shows the accuracy of LLMs across six different tasks (see Section C.2 for additional problems). The top row shows three BAPO-easy problems and the bottom row three BAPO-hard problems (cf. Table 1). Performance is low or rapidly dropping for BAPO-hard problems; in particular, there is no LLM that performs well for all n . In contrast, most LLMs perform consistently well across all n on BAPO-easy problems, with MATCH2 appearing the hardest. We suspect that representational issues interfere in this setting, as the LLM needs perfect understanding of integers.

Comparing models of different scales (solid vs. dashed lines), we can see that in line with the typical observations, larger models appear to perform better overall. However, even with increased scale, no model is able to avoid the degradation our BAPO framework predicts.

4.2 Chain-of-thought reasoning helps on BAPO-hard problems

Given how powerful BAPO-CoTs are in theory, the obvious question is whether CoT improves performance on BAPO-hard tasks. To test this, we prompted each LLM to perform CoT before producing the answer, with a soft limit of 250 words. We also tested o3 [27] and Gemini 2.5 Flash [17], which use (potentially many) internal chain of thought tokens. As Figure 4 shows, CoT modestly improves non-reasoning LLM performance on BAPO-hard problems for smaller input sizes n . The fact that issues still persist indicates that these LLMs may not be applying good low-

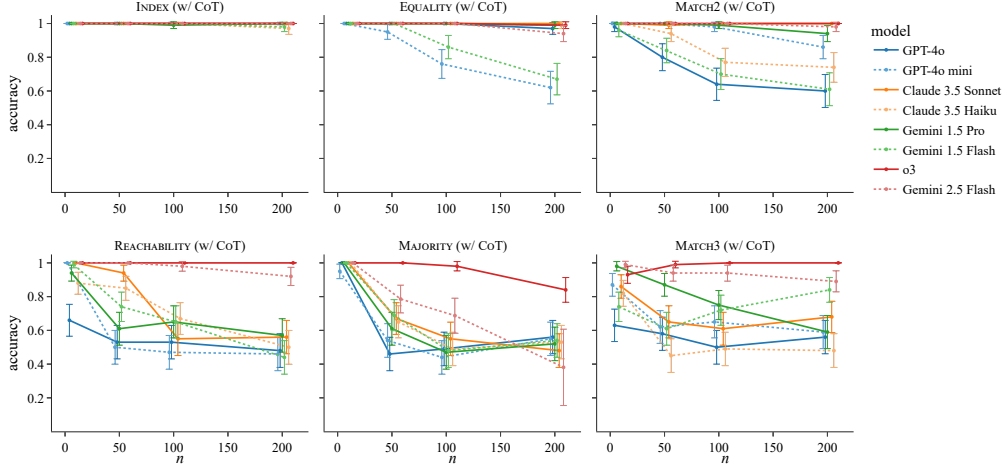


Figure 4: Adding CoT can help LLMs do better on BAPO-hard problems, but substantial performance drops still occur with limited CoT budget (soft limit of 250 words for non-reasoning models). Without imposing a limit on their internal reasoning, o3 and, to a lesser extent, Gemini 2.5 Flash perform extremely well (see Section C.1.2 for their CoT token counts, often in the thousands).

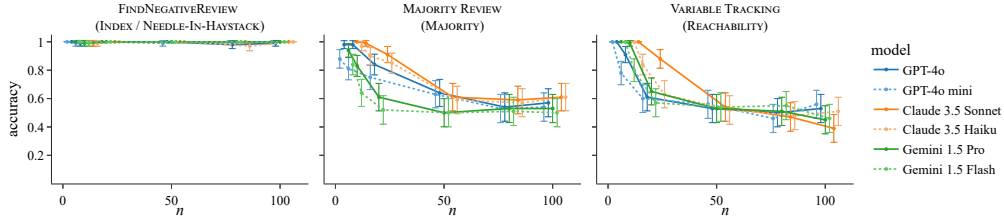


Figure 5: There is good evidence that BAPO-difficulty translates to real-world settings. LLMs can solve real-world tasks that contain BAPO-easy problems (left plot) with much greater accuracy than BAPO-hard problems (two plots on the right).

bandwidth CoT procedures, or that they may require more reasoning tokens. Indeed, without the limit on CoT tokens, o3 and Gemini 2.5 Flash succeed on BAPO-hard problems. This is likely due to the much larger number of CoT tokens they use (over 10k in some cases; see Section C.1.2) and the fact that they are (presumably) trained to use CoT tokens effectively.

4.3 BAPO hardness in real-world tasks

We finally turn to a set of experiments that examine real-world tasks corresponding to BAPO problems. For the first domain, inspired by the ZeroScrolls benchmark [37], we consider hotel reviews from the SPACE dataset [2] and either ask the LLM to find a negative review in a collection of positive reviews (analogous to the BAPO-easy INDEX problem) or decide whether the majority of reviews are positive (analogous to the BAPO-hard MAJORITY problem). We ensure that a baseline LLM can determine the sentiment of each review. The second domain is programming, where we define two chains of assignments and ask whether the final variable has value "a" or "b", extending the variable tracking task from the RULER benchmark [20]. This task is a special case of REACHABILITY, one of our BAPO-hard problems. See Section C.1.3 for experiment and data details. The plots in Figure 5 show that again, BAPO-hardness is a good predictor of LLM performance.

5 Related work

Theoretically, limitations of transformers have been studied via communication complexity [34, 28], circuit analysis [23, 39] and parallel computation frameworks [34, 36], among other methods. We

contribute to this toolbox with the BAPO model, which provides a natural way to study how causal attention exacerbates limits on information flow in LLM architectures.

Among the best known results on transformer expressivity is that transformers are in log-space uniform TC^0 (no more powerful than constant-depth threshold circuits) [23, 39]. Strobl et al. [40] survey the known theoretical results showing an inherent expressivity gap for transformers to recognize formal languages involving unbounded counting or recursion for sufficiently long inputs. For instance, Hahn [18] show that transformers with one-hot hard attention cannot solve PARITY and 2DYCK, whereas Bhattamishra et al. [6] show that a two-layer transformer can solve INDEX, EQUALITY, and DISJOINTNESS. RASP [44] offers a higher-level way to establish similar upper bounds by constructing a transformer as a program, although without causal masking. Some expressivity analyses depend on the size of the transformer: Sanford et al. [34], Fagnou et al. [14] derive logarithmic lower bounds on the number of transformer layers required for graph and entity tracking tasks (related to similar bounds for map-reduce [31]), while Sanford et al. [33] show that a single transformer layer can efficiently reason over pairs of tokens (MATCH2) but not triplets (MATCH3). However, transformer expressivity does not always align with empirical observations of LLM performance. Rather than characterizing theoretical expressivity, the BAPO model captures high-level information flow, which we hypothesize underlies many problem-solving failures in practice.

Even when a task is, in principle, solvable by a transformer, it might still be hard to learn [8]. Edelman et al. [12] argue that attention in transformers tends to represent dependencies among only a small number of tokens, causing failures for global problems. Hahn and Rofin [19] show that although PARITY is representable, it is hard to learn a length-generalizing solution because the training loss landscape is highly sensitive to all the inputs. Similarly, Thomm et al. [42] show that LLMs are data inefficient over compositional problems and Liu et al. [22] argue that transformers learn shortcuts to simulating finite-state automata. Our BAPO model captures this representability-learnability gap by positing that although architecturally the communication bandwidth from prefix tokens of a transformer can be large, the effective communication bandwidth in LLMs is very limited.

Lastly, the success of CoT [43] has been also been analyzed theoretically. Merrill and Sabharwal [24] showed that CoT makes transformers Turing-complete, and Feng et al. [15] argue that CoT enables transformers to solve dynamic programming problems that bounded-depth transformers cannot. Our theory shows that CoT has another benefit, as it dramatically lowers bandwidth requirements: any Turing machine can be simulated by a constant-bandwidth BAPO-CoT.

6 Discussion

The fact that the effective bandwidth of LLMs appears to be so small despite their massive size suggests that simply adding more layers, attention heads, or embedding dimensions might not translate directly to higher BAPO bandwidth. We still lack a full understanding of the root causes of this severely limited bandwidth. It might even be a feature rather than a bug of LLMs: low effective bandwidth may aid flexible and generalizable next-token prediction, and there could be a tradeoff between generalization ability (required for natural language) and exact representation of inputs (required for global reasoning tasks).

Beyond the precise mathematical framework of BAPOs, applying the intuition that lower-bandwidth problems are easier for LLMs can help us understand their successes. For instance, many in-context learning tasks can be solved by a k -nearest neighbor approach, matching a new instance to a small number of in-context examples. This is a procedure whose bandwidth requirement does not scale with the number of in-context examples, but whose accuracy does; this provides a possible explanation for the success of LLMs on such tasks. As another example, needle-in-a-haystack tasks commonly used to benchmark LLMs also require a small amount of cross-stream communication.

Lowering bandwidth. Our model also deepens the understanding of CoT by proving that it reduces the communication requirements of problems—although the number of reasoning steps can be impractically large. This motivates future work to take better advantage of the bandwidth-lowering benefits of CoT or directly optimizing for low bandwidth as part of the training objective when fine-tuning on reasoning chains. Our work also motivates the investigation of methods beyond inference time scaling for reducing the communication burden of problems. For some problems, preprocessing such as simplifying inputs [45] or retrieval [21] may reduce communication load.

BAPO variants. Another future direction is in exploring variations of the BAPO model that could help it align even more closely with the behavior of LLMs. For instance, one limitation of BAPOs that seems overly restrictive is an inability to link information across neighboring tokens. This makes the induction heads task, thought to be an important mechanism in LLMs [25], challenging for BAPOs (we conjecture it to be BAPO- Σ -hard). To address this limitation, we could augment the BAPO model with another round of attention, modeling attention in higher layers. That is, we would have g_1 and g_2 , where g_1 functions like the standard BAPO attention function, while g_2 receives as input the tokens G_1 selected by g_1 (as well as the inputs to g_1). This would allow g_2 to attend to a token that follows another, implementing an induction head. We conjecture that with a constant number of additional attention passes, problems like REACHABILITY and MATCH 3_n would remain hard. This requirement of a second layer of processing for induction heads is analogous to known results that the problem is efficiently solvable by two-layer but not one-layer transformers [7, 35] (although, as we have seen, BAPOs in general are a good model for problem-solving by LLMs with many layers).

Limitations. See Section 2 for discussion of the ways in which the BAPO model does not faithfully represent transformer computation. It also does not capture all failure modes of LLMs, such as tokenization-driven errors; thus, a task being BAPO-easy does not guarantee that LLMs will be able to solve it. Additionally, as discussed above, we do not know the root cause of effective bandwidth limits on LLMs. Finally, many of our lower bounds are loose, and there are many important problems whose BAPO bandwidths have yet to be explored.

7 Conclusions

We introduced the BAPO model of bandwidth-limited computation, designed to quantify and analyze hypothesized limits on the cross-stream communication of transformer-based LLMs. On the theoretical side, we categorize a variety of problems as BAPO-easy, requiring only constant bandwidth, and BAPO-hard, requiring super-constant bandwidth. This dividing line aligns well with problems that modern trillion-parameter-scale LLMs consistently struggle with, supporting the hypothesis that they are constrained in their internal communication and indicating that their effective bandwidth for problem-solving is a small constant. For practitioners, the BAPO framework offers a new lens through which they can view their LLM tasks, possibly opting for mitigation strategies such as tool calling and reasoning in cases where they suspect failures due to BAPO-hardness. Understanding that limited communication bandwidth is at the heart of why LLMs fail to reason globally also unlocks a new set of directions for future work, such as different architectures, reasoning algorithms, or training paradigms.

Acknowledgments

We are grateful to Doug Burger, Philippe Laban, Suriya Gunasekar, Daniel Hsu, Besmira Nushi, Siddharth Suri, Dawen Liang, Harald Steck, Chinmaya Kausik, Nathan Kallus, the MSR AI Interaction and Learning group, and the Netflix Machine Learning Inference Research group for helpful discussions and feedback.

References

- [1] Emmanuel Abbe, Samy Bengio, Aryo Lotfi, Colin Sandon, and Omid Saremi. How far can transformers reason? The globality barrier and inductive scratchpad. In *NeurIPS*, 2024.
- [2] Stefanos Angelidis, Reinald Kim Amplayo, Yoshihiko Suhara, Xiaolan Wang, and Mirella Lapata. Extractive opinion summarization in quantized transformer spaces. *TACL*, 9, 2021.
- [3] Anthropic. The Claude 3 model family: Opus, Sonnet, Haiku. <https://www.anthropic.com/news/claude-3-family>, 2024. Accessed: 2025-04-17.
- [4] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

- [5] Satwik Bhattamishra, Kabir Ahuja, and Navin Goyal. On the ability and limitations of transformers to recognize formal languages. In *EMNLP*, 2020.
- [6] Satwik Bhattamishra, Michael Hahn, Phil Blunsom, and Varun Kanade. Separations in the representational capabilities of transformers and recurrent architectures. In *NeurIPS*, 2024.
- [7] Alberto Bietti, Vivien Cabannes, Diane Bouchacourt, Herve Jegou, and Leon Bottou. Birth of a transformer: A memory viewpoint. In *NeurIPS*, 2023.
- [8] David Chiang and Peter Cholak. Overcoming a theoretical limitation of self-attention. In *ACL*, 2022.
- [9] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [10] Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang (Lorraine) Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena Hwang, Soumya Sanyal, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *NeurIPS*, 2023.
- [11] Mohammad Reza Ebrahimi, Sunny Panchal, and Roland Memisevic. Your context is not an array: Unveiling random access limitations in transformers. In *COLM*, 2024.
- [12] Benjamin L Edelman, Surbhi Goel, Sham Kakade, and Cyril Zhang. Inductive biases and variable creation in self-attention mechanisms. In *ICML*, 2022.
- [13] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. <https://transformer-circuits.pub/2021/framework/index.html>.
- [14] Erwan Fagnou, Paul Caillon, Blaise Delattre, and Alexandre Allauzen. Chain and causal attention for efficient entity tracking. In *EMNLP*, 2024.
- [15] Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: A theoretical perspective. In *NeurIPS*, 2023.
- [16] Google. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- [17] Google. Gemini 2.5 flash. <https://deepmind.google/technologies/gemini/flash/>, 2025. Accessed: 2025-05-14.
- [18] Michael Hahn. Theoretical limitations of self-attention in neural sequence models. *TACL*, 8, 2020.
- [19] Michael Hahn and Mark Rofin. Why are sensitive functions hard for transformers? In *ACL*, 2024.
- [20] Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *COLM*, 2024.
- [21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.
- [22] Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. In *ICLR*, 2023.
- [23] William Merrill and Ashish Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. *TACL*, 11, 2023.

- [24] William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. In *ICLR*, 2024.
- [25] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.
- [26] OpenAI. GPT-4 technical report. *arXiv preprint arxiv:2303.08774*, 2023.
- [27] OpenAI. Introducing o3 and o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>, 2025. Accessed: 2025-05-14.
- [28] Binghui Peng, Srin Narayanan, and Christos Papadimitriou. On limitations of the transformer architecture. In *COLM*, 2024.
- [29] Jacob Pfau, William Merrill, and Samuel R Bowman. Let’s think dot by dot: Hidden computation in transformer language models. In *COLM*, 2024.
- [30] Tim Roughgarden. Communication complexity (for algorithm designers). *Foundations and Trends in Theoretical Computer Science*, 11, 2016.
- [31] Tim Roughgarden, Sergei Vassilvitskii, and Joshua R Wang. Shuffles and circuits (on lower bounds for modern parallel computation). *JACM*, 65, 2018.
- [32] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *TACL*, 9, 2021.
- [33] Clayton Sanford, Daniel J Hsu, and Matus Telgarsky. Representational strengths and limitations of transformers. In *NeurIPS*, 2023.
- [34] Clayton Sanford, Bahare Fatemi, Ethan Hall, Anton Tsitsulin, Mehran Kazemi, Jonathan Halcrow, Bryan Perozzi, and Vahab Mirrokni. Understanding transformer reasoning capabilities via graph algorithms. In *NeurIPS*, 2024.
- [35] Clayton Sanford, Daniel Hsu, and Matus Telgarsky. One-layer transformers fail to solve the induction heads task. *arXiv preprint arXiv:2408.14332*, 2024.
- [36] Clayton Sanford, Daniel Hsu, and Matus Telgarsky. Transformers, parallel computation, and logarithmic depth. In *ICML*, 2024.
- [37] Uri Shaham, Maor Ivgi, Avia Efrat, Jonathan Berant, and Omer Levy. ZeroSCROLLS: A zero-shot benchmark for long text understanding. In *EMNLP*, 2023.
- [38] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 2013.
- [39] Lena Strobl. Average-hard attention transformers are constant-depth uniform threshold circuits. *arXiv preprint arXiv:2308.03212*, 2023.
- [40] Lena Strobl, William Merrill, Gail Weiss, David Chiang, and Dana Angluin. What formal languages can transformers express? A survey. *TACL*, 12, 2024.
- [41] Sainbayar Sukhbaatar, Édouard Grave, Piotr Bojanowski, and Armand Joulin. Adaptive attention span in transformers. In *ACL*, 2019.
- [42] Jonathan Thomm, Giacomo Camposampiero, Aleksandar Terzic, Michael Hersche, Bernhard Schölkopf, and Abbas Rahimi. Limits of transformer language models on learning to compose algorithms. In *NeurIPS*, 2024.
- [43] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

- [44] Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers. In *ICML*, 2021.
- [45] Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik Chaudhari, George Karypis, and Huzefa Rangwala. AgentOccam: A simple yet strong baseline for LLM-based web agents. In *ICLR*, 2025.

A Full problem definitions

Definition 3. DISJOINTNESS: $\{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ is the problem of finding if two sets represented as bitstrings are disjoint, with $\text{DISJOINTNESS}(x, y) = \bigwedge_{i \in [n]} \neg(x_i \wedge y_i)$. We encode a DISJOINTNESS instance (x, y) with the string $x|y$ over $\Sigma = \{0, 1, |\}$.

Definition 4. EQUALITY: $\{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ is the problem of finding if two bitstrings are equal, with $\text{EQUALITY}(x, y) = 1[x = y]$. We encode an EQUALITY instance (x, y) with the string $x|y$ over $\Sigma = \{0, 1, |\}$.

Definition 5. INDEX: $\{0, 1\}^n \times [n] \rightarrow \{0, 1\}$ is the problem of identifying the bit at a given index into the input, with $\text{INDEX}(x, i) = x_i$. We encode an INDEX instance (x, i) with the string xi over $\Sigma = \{0, 1\} \cup [n]$.

Definition 6. REACHABILITY: $([n] \times [n])^m \times [n] \times [n] \rightarrow \{0, 1\}$ is the problem of determining if there is path from s to t in a directed graph G with n nodes and m edges. To encode the problem with perfect tokenization, let $\Sigma = [n] \times [n] \cup [n]$ where the token $(i, j) \in \Sigma$ represents edge (i, j) and the integer tokens represent nodes. An instance of REACHABILITY is specified by the edge list of G in arbitrary order followed by the nodes s and t .

Definition 7. MAJORITY: $\{0, 1\}^n \rightarrow \{0, 1\}$ is the problem of determining whether the input has strictly more ones than zeros, with $\text{MAJORITY}(x) = 1 \left[\sum_{i \in [n]} x_i > n/2 \right]$.

Definition 8. Given input $x \in \mathbb{Z}_m^n$, MATCH 3_n is the problem of determining whether there are some $i, j \in [n]$ such that $x_n + x_i + x_j \equiv 0 \pmod{m}$. Additionally, MATCH 2_n is the problem of determining whether there is some $i \in [n]$ such that $x_n + x_i \equiv 0 \pmod{m}$. We encode instances of these problems with the string x over $\Sigma = \mathbb{Z}_m$.

Definition 9. UNIQUE: $\Sigma^* \rightarrow \Sigma \cup \{\emptyset\}$ is the problem of identifying any unique token in the input. That is, $\text{UNIQUE}(x_1 \dots x_n) = x_i$ s.t. $\sum_{j=1}^n 1[x_j = x_i] = 1$, or $\emptyset$ if no such x_i exists.

Definition 10. SETDIFF: $\Sigma^* \times \Sigma^* \rightarrow \Sigma \cup \{\emptyset\}$ is the problem of identifying any token in the first part of the input that doesn't appear in the second part. That is, $\text{SETDIFF}(c_1 \dots c_n, d_1 \dots d_m) = c_i$ for some $i \in [n]$ such that $c_i \neq d_j$ for all $j \in [m]$, or $\emptyset$ if no such element exists. We encode a SETDIFF instance $(c, d) \in \Sigma^* \times \Sigma^*$ with the string $c|d$ over $\Sigma \cup \{|\}$.

B Proofs

Proof of Theorem 1. For INDEX instances given as $s = xi$, the idea is simple: we set $g(s_{k+1} \dots s_n, k, s_j, j) = 1$ if and only if $j = s_n$. The suffix oracle returns whatever token is attended to—unless the index is in the suffix (i.e., $s_n \geq k + 1$), in which case the suffix oracle immediately returns s_j . No output from f is required.

For a DISJOINTNESS instance encoded as $s = x|y$, the key is that we only need a single counterexample bit where $x_i = y_i = 1$ to conclude that x and y are not disjoint. If no such counterexample exists, then they are disjoint. We can construct a $(1, 1)$ -BAPO using this fact. The prefix oracle outputs 0 (“no counterexample found”) if (a) the token $|$ is not in the prefix or (b) if the token $|$ is in the prefix and the starting bits of y present in the prefix are disjoint with the corresponding bits of x . If there is a counterexample bit in y visible in the prefix, the prefix oracle outputs 1. If the suffix oracle receives a 1 from f , it always outputs 0 indicating x and y are not disjoint. Meanwhile, the attention function attends to any prefix bits that would be a counterexample to the disjointness of x and y ; i.e., $g(s_{k+1} \dots s_n, k, s_i, i) = 1$ if and only if either (a) $|$ is not in the suffix and $i' = i + (k + |s_{k+1} \dots s_n| - 1)/2 + 1 \geq k + 1$ (i.e., the token we need to compare to s_i is in the suffix) and $s_{i'} = s_i = 1$ or (b) $|$ is in the suffix and $s_{i'} = s_i = 1$. If the attended token set G is nonempty, then x and y are not disjoint and the suffix oracle outputs 0. If the attended token set is empty and $|$ is in the suffix, the suffix oracle can check the portion of x present in the suffix against the corresponding bits of y . If no counterexample bits are found (by f , g , or h), then none exists and the suffix oracle outputs 1.

For EQUALITY, the same idea applies: we only need one counterexample $x_i \neq y_i$ to conclude that $x \neq y$. The construction is precisely the same as for DISJOINTNESS, except we look for counterexamples where $x_i \neq y_i$ rather than $x_i = y_i = 1$. $\square$

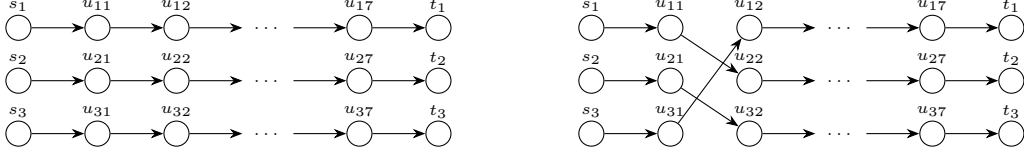


Figure 6: Left: the graph P from the proof of Theorem 2 for $p = c = 3$, so $n = p^c = 27$ and $m = p^c - p = 24$. Right: P with the permutation $\pi = 231$ applied to the targets of the second-layer edges. Applying any non-identity permutation to a single layer changes the connectivity of at least one s - t pair.

Proof of Theorem 2. Suppose such a BAPO exists for a contradiction, and let f and g be its prefix oracle and attention function, with bandwidths $a = o(m^{1/c} \log m)$ and $b = o(m^{1-2/c})$, respectively. We will construct a family of prefixes and suffixes where the prefixes contain the entire graph and the suffixes contain the nodes s and t , taking care that the graphs in different prefixes are mostly identical to saturate attention.

Let $n = p^c$ for some integer p for simplicity (if n is not expressible as an c th power of an integer, the argument would involve various ceilings or floors, but these do not affect the asymptotics). Let P be the graph consisting of p disjoint directed paths with start nodes $s_1, \dots, s_p$ and target nodes $t_1, \dots, t_p$, where each path has exactly p^{c-1} nodes. Then, for each $i \in [p]$ and $j \in [p^{c-1} - 2]$, let u_{ij} be the unique node at distance j from s_i . See Figure 6 for a visualization of this graph with $p = c = 3$. We will keep these node labels fixed but modify P by permuting edges based on the attention function g to render attention useless. Note that we can apply a permutation $\pi \in S_p$ to the target nodes of the edges departing from $u_{1j}, \dots, u_{pj}$ while maintaining the property that P is a disjoint collection of length p^{c-1} paths each starting from an s node and ending at a t node, although the permutation changes which s - t pairs are connected (see Figure 6, right). To construct our shared prefixes that saturate the attention function g , we will need to take care to place edges at consistent indices in the prefixes. To this end, we fix the edge order to first list the edges departing from $s_1, \dots, s_p$, then $u_{11}, \dots, u_{p1}$, then $u_{12}, \dots, u_{p2}$, etc. Call this the canonical edge order and let $I(u)$ denote the index in the canonical order where the edge departing u is placed. Note that the length of the full edge list is $m = p^c - p$. Given these observations, construct the shared prefix graph P^* as follows:

1. Initialize P^* to have the same set of nodes as P . Initialize S to be the set of all s_i and u_{ij} nodes, which will store the current set of nodes that still need an outgoing edge.
2. For each pair $(i, j) \in [p] \times [p]$:
 - (a) For $\ell = 1, \dots, b$:
 - i. If there is some node u in S and some node v in the layer to the right of u such that $g(s_i t_j, m, I(u), (u, v)) = 1$ and v has indegree 0 in P^* : add (u, v) to P^* and remove u from S .
Check every feasible edge we could add; if g wants to attend to any edge (u, v) , add it to P^ .*
 - ii. Else: exit the inner for loop.
If there are no remaining edges g would attend to given suffix $s_i t_j$, then we can stop, as attention is now saturated for this suffix.
3. Let E^* be the set of edges in P^* at this point in the algorithm. These will be shared among all prefixes to saturate attention. To complete the paths in P^* arbitrarily, connect each node with outdegree 0 (that is not a t node) to the first node with indegree 0 in the next layer.

This procedure produces a graph P^* which can be reached by starting with P and permuting the targets of the edges in each layer by some π , since each edge we add connects a node with current outdegree 0 to a node in the next layer with current indegree 0. That is, P^* is also a disjoint collection of p paths of length p^{a-1} each connecting some s_i to some t_j . We have placed edges in P^* such that for any suffix $s_i t_j$ with $i, j \in [p]$, there exists some attended token set G where *every* edge in G is in

E^* , so we can modify the other edges in P^* without alerting the attention. Each iteration of the outer for loop ensures one suffix has its attention masked; if we complete b iterations of the for loop, then we have ensured there is some G with $|G| = b$ that contains only edges in E^* , and if we need to exit early due to the nonexistence of an attended edge in line 2(a)i, then no other edges we end up placing in P^* will ever be attended to given that suffix (as 2(a)i checks *every* feasible edge). Moreover, the total number of edges in E^* is at most $bp^2 = o(m^{1-2/c}p^2) = o((p^c)^{1-2/c}p^2) = o(p^c)$ (since there are $m = p^c - p < p^c$ edges in P^*).

Now, since there are $p^{c-1} - 1$ layers of edges in P^* , there must be some layer with $\Theta(p)$ edges not contained in E^* —otherwise we would be able to find $\Theta(p^c)$ edges in E^* , a contradiction. Call this layer j^* and let $d = \Theta(p)$ be the number of edges in layer j^* that are not in E^* . Construct a family of graphs $\mathcal{P}$ of size $d!$ by taking P^* and applying each permutation $\pi \in S_d$ to the targets of edges in layer j^* that are not in E^* . We now have our full collection of prefixes and suffixes: the $d!$ prefixes are the canonical edge lists of each graph in $\mathcal{P}$ and the p^2 suffixes are every $s_i t_j$ pair.

By Stirling's approximation and the fact that $d = \Theta(p)$, $\log_2(d!) = \Theta(d \log d) = \Theta(p \log p)$. Thus, since f has bandwidth $o(m^{1/c} \log m) = o(p \log p)$, f must have a collision on the size $d!$ family of prefixes generated by $\mathcal{P}$ (as it takes $\log_2(d!) = \Theta(p \log p)$ bits to distinguish between the prefixes). These two colliding graphs are generated by different permutations on the edges in layer j^* , so there is some pair of start and target nodes s^*-t^* connected in one of the graphs but not the other (as the edges in all other layers are identical). Given the suffix s^*t^* , we find that the BAPO fails to distinguish between the colliding graphs: it gets the same output from f and there is some attended token set G that contains only edges in E^* , which are identical in the two graphs. Given this adversarial G containing only attention-saturating edges, the suffix oracle h gets identical inputs: the same output from f , the same G , the same suffix s^*t^* and the same prefix size m . In this case, the BAPO fails to solve the problem in one of the two graphs, as one has an s^*-t^* path and the other doesn't. $\square$

Proof of Theorem 3. First, a $(\lceil \log_2 n \rceil, 0)$ -BAPO can solve MAJORITY by having $f(x)$ output the number of 1s in x , which takes at most $\lceil \log_2 n \rceil$ bits and requires no attention tokens.

Now suppose for a contradiction that (f, g, h) is a BAPO with prefix bandwidth $a = o(\log n)$ and attention bandwidth $b = o(n^{1-\epsilon})$ that solves MAJORITY on inputs of length n . Let m be any positive integer and let $\ell = m^{(1-\epsilon)/\epsilon}$ (for simplicity assume ℓ is an integer; otherwise we could take a ceiling without disrupting the proof). The proof follows three high-level steps: (1) set up the structure of prefix-suffix pairs, (2) design the prefixes so that attention cannot distinguish between them, (3) find a colliding pair of prefixes and suffixes that the BAPO cannot distinguish between but which have different MAJORITY answers.

Given any $\pi \in S_m$ (the permutations on $[m]$), define $X_\pi = \{(\pi(0^\ell m 1^\ell m 0^k 1^{m-k}), 0^{m-k-1} 1^{k+1})\}_{k=0}^{m-1}$. Note that every prefix-suffix pair in X_π has total length $n = 2(\ell+1)m = 2m^{1+(1-\epsilon)/\epsilon} + 2m$, with $(\ell+1)m + 1$ ones and $(\ell+1)m - 1$ zeros; so all of them have a majority of ones. Let s_k be the suffix in X_π with $k+1$ ones and let $S = \{s_k\}_{k=0}^{m-1}$. We'll show to pick π based on g such that X_π fools the BAPO. We'll give the BAPO $\ell = m^{(1-\epsilon)/\epsilon}$ attention tokens. Note that $m^{(1-\epsilon)/\epsilon} = \Theta(n^{1-\epsilon})$, as $(m^{1+(1-\epsilon)/\epsilon})^{1-\epsilon} = m^{(1-\epsilon)/\epsilon}$.

For any suffix s , define $\Gamma_0(s) = \{i \in [(2\ell+1)m] : g(s, (2\ell+1)m, 0, i) = 1\}$ to be the set of prefix indices where g selects 0s given suffix s , and define $\Gamma_1(s)$ similarly to be the set of prefix indices where g selects 1s. We'll pick π so that for every $s \in S$, π permutes the prefixes so that it places leading zeros and ones in at least ℓ indices selected by g (or such that g selects only indices masked by leading zeros and ones) for every suffix in X_π . We can construct such a π with the following procedure.

1. Initialize $\Pi_0 = \emptyset$ and $\Pi_1 = \emptyset$.
These sets store indices to place leading zeros and ones so that we fool g for all suffixes.
2. For $k = 0, \dots, m-1$:
 - (a) While $|\Gamma_0(s_k) \cap \Pi_0| + |\Gamma_1(s_k) \cap \Pi_1| < \ell$:
If this is false, we have succeeded in masking $\geq \ell$ positions selected by g given s_k .

- i. If $\exists i \in \Gamma_0(s_k) \setminus (\Pi_0 \cup \Pi_1)$: add i to Π_0 .
Mask a position where g selects a zero.
 - ii. Else if $\exists i \in \Gamma_1(s_k) \setminus (\Pi_0 \cup \Pi_1)$: add i to Π_1 .
Mask a position where g selects a one.
 - iii. Else: exit the while loop.
There are no unmasked positions selected by g on s_k , so we have succeeded on s_k .
3. Let $i = 1$. For each $j \in \Pi_0$, set $\pi(i) = j$ and increment i .
Make π permute leading zeros into the masking indices we have picked.
 4. Let $i = \ell m + 1$. For each $j \in \Pi_1$, set $\pi(i) = j$ and increment i .
Make π permute leading ones into the masking indices we have picked.
 5. Fill in π where not yet defined with the remaining indices in order.

This procedure terminates, since each iteration of the while loop increases the combined sizes of the intersections in (a) by 1 or exists the while loop. Additionally, it generates a valid permutation and Π_0 and Π_1 are disjoint, since we only ever add indices i which are in neither of them to exactly one of them. Next, for every $(\pi(p), s) \in X_\pi$, either: (1) there are at least ℓ indices I selected by g on input $(\pi(p), s)$ with $I \subseteq \Pi_0 \cup \Pi_1$ or (2) on input $(\pi(p), s)$, g selects only indices in $\Pi_0 \cup \Pi_1$. Case (1) occurs if we are able to complete the while loop for s without ever hitting the else in iii, since then we will have ℓ such indices in Π_0 and Π_1 . In 3, we ensure the indices where g wishes to select zeros are masked with leading zeros; in 4, we ensure the indices where g wishes to select ones are masked with leading ones. Case (2) occurs if we hit the else in iii, as in that case, there are no prefix indices selected by g which are not already in $\Pi_0 \cup \Pi_1$ and thus all selected indices will be masked by leading zeros and ones.

This π gives us a set X_π where the set of attended tokens G is entirely useless over *every* prefix-suffix combination, in the worst case over adversarial choices of which ℓ tokens are attended to. That is, for any prefix $\pi(p)$ and any suffix s from X_π that may be from different pairs, $G_{(\pi(p), s)}$ (the attended token set for the given prefix-suffix combination) can be identical: an adversary picking which ℓ tokens are attended to can pick all ℓ of them to come from indices masked by leading zeros and ones (i.e., indices from $\Pi_0 \cup \Pi_1$), which are identical across all m prefixes in X_π , as the prefixes only differ in indices not in $\Pi_0 \cup \Pi_1$.

Now, there are m distinct strings in X_π . Since the prefix bandwidth of f is $o(\log n) = o(\log m^{1+(1-\epsilon)/\epsilon}) = o(\log m)$, there are fewer than $2^{\log_2 m} = m$ distinct outputs of f , so by the pigeonhole principle there are two elements $(a, b), (c, d) \in X_\pi$ where $f(a) = f(c)$. If (a, b) has $k + 1$ ones in b and (c, d) has $k' + 1$ ones in d (without loss of generality assuming $k' > k$), then the string ab has a majority of ones ($\ell m + m - k + k' + 1 > (\ell + 1)m + 1$ ones total) but cb does not ($\ell m + k' + m - k - 1 \geq (\ell + 1)m$ zeros total). Thus, consider the inputs (a, b) and (c, b) . The prefix oracle f outputs the same thing in both cases, and as we have seen that $G_{(a, b)}$ can equal $G_{(c, b)}$. Moreover, the suffix oracle gets the same suffix in both cases, so there exist attended token sets where the suffix oracle outputs the same answer on both input pairs (having received all the same inputs: same prefix oracle output, same suffix, same attended tokens, same split index). But (a, b) and (c, b) have opposite answers to MAJORITY: (a, b) has a majority of ones and (c, b) does not. Thus the BAPO answers one of these instances incorrectly for some attended token set. $\square$

While near-linear attention does nothing to help the required asymptotic prefix bandwidth over zero attention, our proof technique does result in a weaker lower bound on prefix bandwidth with even more attention.

Corollary 1. *No $(o(\log \log n), o(n/\log n))$ -BAPO can solve MAJORITY on length n inputs.*

Proof. Now we use even more masking bits, $m2^m$ zeros and ones, so $n = m2^{m+1} + 2m$. With $2^m = \Theta(n/\log n)$ attention tokens, using the same argument shows that $o(\log \log n) = o(\log m)$ prefix bandwidth can cause a collision. $\square$

We suspect that this lower bound is loose, and that in fact $\log n$ prefix bandwidth is still required even with $\Theta(n/\log n)$ attention. However, we can increase the attention bandwidth even more (all the way to $\Theta(n)$, but still less than n) so that we do finally lower the asymptotic prefix bandwidth

requirement. For instance, with $n - \log n$ attention tokens, we only need $\Omega(\log \log n)$ prefix bandwidth.

Proposition 1. *For any $a(n) = o(n)$, there is a $(\lceil \log_2 a(n) \rceil, n - a(n))$ -BAPO solving MAJORITY on length n inputs.*

Proof. The BAPO is simple: g attends to the first $n - a(n)$ tokens of the prefix and f passes on the ones count in the remaining part of the prefix, which takes at most $\lceil \log_2 a(n) \rceil$ bits. This information suffices to allow the suffix oracle to solve the problem. $\square$

The hardness of MAJORITY immediately implies at least the same degree of hardness for MEDIAN, the problem of finding the median of a input sequence of integers, and MODE, the problem of finding the most frequent item in a stream. We believe these bounds are very loose.

Corollary 2. *No $(o(\log n), O(n^{1-\epsilon}))$ -BAPOs solve MEDIAN or MODE on length n inputs for any $0 < \epsilon < 1$.*

Proof. On bitstrings, the median and mode are 1 if and only if the string has a majority of 1s. $\square$

For proving MATCH3_n is BAPO-hard, the following lemma helps us find x_i - x_j pairs that form a match with some particular suffix $s = x_n$, but such that x_i and x_j do not form matches with any suffix $s \in S$ and other prefix integer $z \in Z$, where Z is the set of integers we have already decided to place in prefixes.

Lemma 1. *Let $S, Z \subset \mathbb{Z}_m$ with $m > 100$, $\max_{s \in S} s \leq \sqrt{m}$, $|S| \leq \sqrt{m}/2$, and $|Z| \leq \sqrt{m}/2$. For every $s \in S$, there exist $x, y \in \mathbb{Z}_m \setminus Z$ s.t.:*

1. $x + y + s \equiv_m 0$,
2. for all $z \in Z$ and all $s \in S$, $x + z + s \not\equiv_m 0$ and $y + z + s \not\equiv_m 0$.

Proof. Condition 2 is satisfied as long as $x, y \in \mathbb{Z}_m \setminus Z$ are not in $-(S + Z) = \{-(s + z) \bmod m : s \in S, z \in Z\}$, which has size at most $(\sqrt{m}/2)^2 = m/4$. This leaves at least $3m/4 - \sqrt{m}/2$ admissible values in $\mathbb{Z}_m \setminus Z$. We'll show that there are enough x - y pairs with distinct values of x and y satisfying Condition 1 that they cannot all be disallowed by Condition 2 and the requirement that $x, y \notin Z$.

Consider all pairs $x, y \in \mathbb{Z}_m$ such that $x + y + s \equiv_m 0$: for each $i \in \mathbb{Z}_m$, we can set $x = i \bmod m$ and $y = -s - i \bmod m$. If we consider $0 \leq i \leq m/4 + \sqrt{m}/2 + 1$, the values x and y take on are all distinct (since $s \leq \sqrt{m}$ and $m/4 > 2\sqrt{m}$, every y value is larger than every x value). Since there are only $m/4 + \sqrt{m}/2$ disallowed values for x and y , there must be some x, y pair in this range of more than $m/4 + \sqrt{m}/2$ values of i where both x and y are admissible. $\square$

Proof of Theorem 4. First, consider MATCH2₂. The attention function g can select any prefix elements x_i such that $x_n + x_i \equiv 0 \pmod{m}$, since x_n is always in the suffix. The suffix oracle only needs a single such example to confirm that this is a yes instance. If the set of attended tokens is empty, then the suffix oracle can check if there are any matches in the suffix. If not, this is a no instance. We do not need f at all.

Now we show that MATCH3_n is BAPO-hard. Suppose for a contradiction that (f, g, h) is a BAPO solving MATCH3_n with prefix bandwidth $o(n/b(n))$ and attention bandwidth $b(n)$. We'll construct a collection of prefixes and suffixes that fools the BAPO with total input length n . Let $n > 10$ and pick $m = n^2$.

Consider the set of suffixes $S = \{i : 0 \leq i < n/(8b(n))\}$. Note that since $b(n) = o(n)$, $|S| = \Theta(n/b(n))$ is growing with n . For a suffix s , let $G(s) = \{(x, i) : i \in [n - 1], x \in \mathbb{Z}_{n^2}, g(s, n - 1, x_i, i) = 1\}$ be the set of integer-index pairs that g attends to given suffix s .

1. Initialize $P^* = \{(\lfloor n^2/3 \rfloor), 1\}$ and $I = \{1\}$
We'll use $\lfloor n^2/3 \rfloor$ as filler; placing it in P^* here ensures it doesn't match any prefix integer.
2. For $s \in S$:

- (a) While $|P^* \cap G(s)| < b(n)$:
- i. If there exists some $(x, i) \in \mathbb{Z}_{n^2} \times ([n-1] \setminus I)$ for which (1) $(x, i) \in G(s)$ and (2) $x + y + s' \not\equiv_{n^2} 0$ for all $s' \in S$ and all $(y, j) \in P^*$: add (x, i) to P^* and add i to I
Add a masking integer to the prefix, but only allow masking integers that don't form a match with existing masking integers and any suffix.
 - ii. Else: break out of the while loop
If there are no integers that g wants to attend to (among those not forming matches with existing masking integers), then we're done with this suffix.

After the procedure, P^* contains at most $b(n)|S| + 1 \leq n/8 + 1$ occupied indices. Let Z be the set of values in P^* , with $|Z| \leq n/8 + 1$. Moreover, we have ensured that no pair of integers in P^* can form a match with any suffix integer s , since we only add x to P^* if for every y in P^* and $s \in S$, $x + y + s \not\equiv_{n^2} 0$. Lastly, if for some suffix s , we were unable to saturate attention and hit the else in 2(a)ii, then no other integers we add to a prefix can be attended to, since we will only be adding integers to prefixes that do not form matches with any integer in P^* and any suffix (and therefore were already checked for attention in 2(a)i).

Now, for each $s \in S$, we find some x_s and y_s using Lemma 1 such that $x_s + y_s + s \equiv_{n^2} 0$, but x_s and y_s do not form matches with any other $z \in Z$ (recall that we have initialized Z to contain all values in P^*). Add each x_s and y_s to Z before the next application of Lemma 1 to ensure that we do not create any matches across x - y pairs. After doing this for each $s \in S$, the final size of Z is $\leq n/8 + 1 + n/(4b(n)) \leq 3n/8 + 1$, so the size limit on Z required by Lemma 1, namely $|Z| \leq \sqrt{m}/2 = n/2$, is always satisfied (since we picked $n > 10$, $n/2 > 3n/8 + 1$). Let $P = \{(x_s, y_s)\}_{s \in S}$ be the x - y pairs we find using this procedure.

For every subset $R \subseteq P$, construct a prefix of length $n-1$ by first filling in all of the masking integers in P^* (filling at most $n/8 + 1$ positions) and then adding in the x and y values in R in arbitrary indices (filling at most $2|S| = n/(4b)$ additional positions). Fill the remaining indices with $\lfloor n^2/3 \rfloor$, which cannot form a match with itself and any suffix integer (since the suffix integers are at most $n/(4b(n))$), and which we already ensured cannot form a match with any integer in P or P^* by placing it in Z . This gives us $2^{|S|}$ distinct prefixes, each of which has matches with a distinct set of suffixes. That is, for any two prefixes $p_1 \neq p_2$, there exists some suffix integer s where $p_1 s$ and $p_2 s$ have opposite answers to $\text{MATCH}_{3,n}$, since there is some pair (x_s, y_s) in one prefix, but not the other, which forms a match with s , while no other pair of integers forms a match with s . But with prefix bandwidth $o(n/b(n)) = o(|S|)$, there is some prefix oracle collision (as there are $2^{o(|S|)}$ distinct outputs of f , too few for the $2^{|S|}$ distinct prefixes). Moreover, these colliding prefixes are indistinguishable to attention for all suffixes, since we have ensured that for every suffix, attention can be saturated by integers in P^* , which are identical across all prefixes. Therefore the BAPO fails to solve the problem. $\square$

Proof of Theorem 5. For the $(2k, 0)$ -BAPO, the prefix oracle transmits two bitstrings of length k , one indicating which elements of Σ appear exactly once in the prefix and one indicating which elements of Σ appear in the prefix one or more times. The suffix oracle outputs (a) any element that appears exactly once in the prefix but not in the suffix, which it finds from the first bitstring; (b) any element that appears exactly once in the suffix but not in the prefix, which it can find from the second bitstring; or (c) $\emptyset$ otherwise. This allows the suffix oracle to correctly solve UNIQUE .²

For the lower bound, suppose for a contradiction that (f, g, h) is an $(o(k/b(k)), b(k))$ -BAPO solving UNIQUE . We will first construct a set of partial prefixes P^* for which the attention mechanism cannot distinguish all following suffixes. This leads to a pair of fooling prefixes with different answers that the BAPO cannot distinguish.

Fix an arbitrary order over the symbols in $\Sigma = \{\Sigma_1, \dots, \Sigma_k\}$. For any $A \subseteq \Sigma$, let $\text{cat}(A)$ denote the string consisting of each token $\Sigma_i \in A$ concatenated in order. Let $b' = 4b(k)$. We'll construct a collection of k/b' suffixes (for convenience, assume k/b' is an integer; otherwise, the construction would involve ceilings and floors, but this this would not affect the argument): let $\sigma_i = \text{cat}(\Sigma \setminus$

²Additionally, there is a clever $(\lceil \log_2 |\Sigma| \rceil, 0)$ -BAPO for the special case of UNIQUE where every element appears an even number of times, except a single unique item: with this restriction, taking the bitwise exclusive or over binary encodings of the tokens solves the problem—but this fails in general.

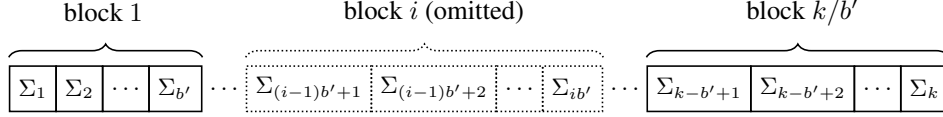


Figure 7: Constructing σ_i involves leaving out the i th block from the vocabulary Σ , leaving $k - b'$ tokens.

$\{\Sigma_{b'(i-1)+j}\}_{j=1}^{b'}$) and $S = \{\sigma_i \sigma_i\}_{i=1}^{k/b'}$. As Figure 7 shows, this means that the partial suffix σ_i is missing the i th block of b' contiguous tokens in Σ , so each full suffix $s_i = \sigma_i \sigma_i$ has length $2(k - b')$ and $|S| = k/b'$ by construction. Note that we have duplicated each σ_i to ensure that no suffix token is unique.

Now, we perform the usual overloading procedure to start building up prefixes of length k that saturate the attention function for every suffix. Let $G(s) = \{(x, i) \in \Sigma \times [k] : g(s, k, x, i) = 1\}$. Perform the following procedure to construct a partial prefix P^* .

```

Initialize  $P^* \leftarrow \emptyset, I \leftarrow \emptyset$ 
for  $s \in S$  do
  while  $|P^* \cap G(s)| < b(k)$  do
    if  $\exists x \in \Sigma, i \in [k] \setminus I$  such that  $(x, i) \in G(s)$  then
      Add  $(x, i)$  to  $P^*$ ; add  $i$  to  $I$ 
    else
      break

```

This procedure results in at most $b(k)|S| = b(k)\frac{k}{b'} = b(k)\frac{k}{4b(k)} = k/4$ positions in P^* being filled with masking symbols. Let Z be the set of unique symbols in P^* , with $|Z| \leq |P^*| \leq k/4$. To ensure none of these are unique, fill in another $\leq k/4$ arbitrary indices in P^* with an additional copy of each symbol in Z . As in the previous proofs, attention has now been rendered useless for every suffix, regardless of what additional symbols we add in remaining prefix indices. Consider the blocks we used to construct the suffixes, namely $\{\Sigma_{b'(i-1)+j}\}_{j=1}^{b'}$ for $i \in [k/b']$ (Figure 7). It must be the case that more than half of the blocks have an element not in Z (suppose this wasn't the case and $> |S|/2$ blocks have all of their elements in Z , then $|Z| > b'|S|/2 = k/2$, contradicting that $|Z| \leq k/4$). Thus, we can find a collection of tokens $Y = \{y_1, \dots, y_{|S|/2}\}$ such that each y_i is in a different block and $Y \cap Z = \emptyset$, as well as one extra filler token y_0 from yet another different block.

For each subset $R \subseteq Y$, construct a prefix p_R by starting with P^* , placing each $y_i \in R$ in an arbitrary unfilled index (so that the prefix now has $\leq k/2 + |S|/2$ filled indices) and filling the remaining indices with copies of y_0 . This gives us $2^{|S|/2} = 2^{k/(2b')}$ distinct prefixes. With $o(k/b(k))$ prefix bandwidth, we only have $2^{o(k/b(k))}$ distinct outputs of f , so there are two prefixes p_R and $p_{R'}$ that collide. These two prefixes have different subsets R and R' of Y values, so there is some y_i in one prefix but not the other (say $y_i \in R, y_i \notin R'$). Consider the suffix s_{-i} that is missing the block of tokens to which y_i belongs.

We observe that:

- s_{-i} has no unique tokens due to its construction
- $[p_{R'}, s_{-i}]$ has no unique tokens, since all of its Y values in the prefix appear in the suffix due to the fact that every element of Y comes from a different suffix block. Thus, every element of Y except y_i appears in s_{-i} .
- $[p_R, s_{-i}]$ only has y_i as a unique token, since (a) all other tokens in Y have a copy in the suffix due to the above fact, (b) the masking and filler tokens in the prefix are all duplicated at least twice

Since the BAPO can observe the same attention set G given both of these strings and receives the same output of f from the two prefixes, it gets one of these instances wrong for some G . $\square$

Proof of Theorem 6. For the $(k, 0)$ -BAPO, the prefix oracle sends a bitstring denoting all elements of Σ appearing in c but not d (as far as it can tell). The suffix oracle can then remove all elements of d in the suffix and compute an answer.

For the lower bound, as usual, suppose for a contradiction that (f, g, h) is an $(o(k/b(k)), b(k))$ -BAPO solving SETDIFF. We will use the a similar collection of omitted-block suffixes as we did for UNIQUE. As we did in that proof, order the elements of Σ and define $\text{cat}(A)$. Let $b' = b(k)/4$. We'll construct a collection of k/b' suffixes (again assuming for convenience that k/b' is an integer): let $\sigma_i = \text{cat}(\Sigma \setminus \{\Sigma_{b'(i-1)+j}\}_{j=1}^{b'})$ and $S = \{\sigma_i\}_{i=1}^{k/b'}$ (recall Figure 7; note that in contrast with UNIQUE, there is no need to duplicate the suffixes).

Now, we perform the usual overloading procedure to start building up prefixes of length $3k + 1$ that saturate the attention function for every suffix. The prefixes we build up will be of the form $c_1 \dots c_k | d_1 \dots d_{2k}$. That is, the prefix-suffix split occurs somewhere in the middle of the d input to SETDIFF. Let $G(\sigma) = \{(x, i) \in \Sigma \times [3k + 1] : g(\sigma, k, x, i) = 1\}$. Perform the following procedure to construct a partial prefix P^* , which we initialize to already contain the divider token $|$.

```

Initialize  $P^* \leftarrow \{(|, k + 1)\}$ ,  $I \leftarrow \{k + 1\}$ 
for  $\sigma \in S$  do
  while  $|P^* \cap G(\sigma)| < b(k)$  do
    if  $\exists x \in \Sigma, i \in [3k + 1] \setminus I$  such that  $(x, i) \in G(\sigma)$  then
      Add  $(x, i)$  to  $P^*$ ; add  $i$  to  $I$ 
    else
      break

```

This procedure results in at most $b(k)|S| = b(k)\frac{k}{b'} = b(k)\frac{k}{4b(k)} = k/4$ positions in P^* being filled with symbols that saturate g . Let Z be the set of unique symbols in P^* , with $|Z| \leq |P^*| \leq k/4$. For each symbol in Z , place another copy of it in P^* on the opposite side of the divider as its first copy. This ensures that no symbols in Z can be SETDIFF answers, while leaving at least $3k/4$ open positions in P^* before the divider and more than k open positions after the divider.

Now, consider the blocks we used to construct the suffixes, namely $\{\Sigma_{b'(i-1)+j}\}_{j=1}^{b'}$ for $i \in [k/b']$ (Figure 7). Just as before, it must be the case that more than half of the blocks have an element not in Z . Thus, we can find a collection of tokens $Y = \{y_1, \dots, y_{|S|/2}\}$ such that each y_i is in a different block and $Y \cap Z = \emptyset$, as well as one extra filler token y_0 from yet another different block. Let $X = \Sigma \setminus (Z \cup Y)$. Place every $x \in X$ in an unfilled index of P^* after the divider (i.e., in the d portion) and fill all remaining empty spots of P^* after the divider with the filler y_0 .

For each subset $R \subseteq Y$, construct a prefix p_R by starting with P^* (which is now full after the divider, but has at least $3k/4$ open positions before the divider), placing each $r \in R$ in an arbitrary unfilled index before the divider, and then filling in all remaining indices with y_0 . As before, this gives us $2^{|S|/2} = 2^{k/(2b')}$ distinct prefixes. With prefix bandwidth $o(k/b(k))$, we only have $2^{o(k/b(k))}$ distinct outputs of f , so there are two prefixes which have identical f outputs, call them p_R and $p_{R'}$. These contain different sets R and R' of Y values; thus, there is some $y_i \in Y$ in one prefix but not the other. Suppose without loss of generality that $y_i \in R, y_i \notin R'$. Consider the suffix σ_i that is missing the block of tokens to which y_i belongs. Notice that:

- $p_R \sigma_i$ has only a single SETDIFF answer, which is y_i : we ensured that every Y element only appears in a prefix before the divider, and y_i does not appear in σ_i . Every other symbol in the missing block of σ_i is in p_R after the divider by construction, and thus is not a valid answer.
- $p_{R'} \sigma_i$ has SETDIFF answer $\emptyset$: the only symbol not appearing after the divider in $p_{R'} \sigma_i$ is y_i , which is not before the divider.

Thus these two instances have different SETDIFF answers, but they are indistinguishable to the BAPO (for some adversarially chosen G) due to the saturation of attention and the f collision. $\square$

Proof of Theorem 7. Let $M = (Q, \{0, 1\}, \Lambda, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ be a Turing machine deciding L using space $s(n)$, using the definition of Turing machines from Sipser [38].

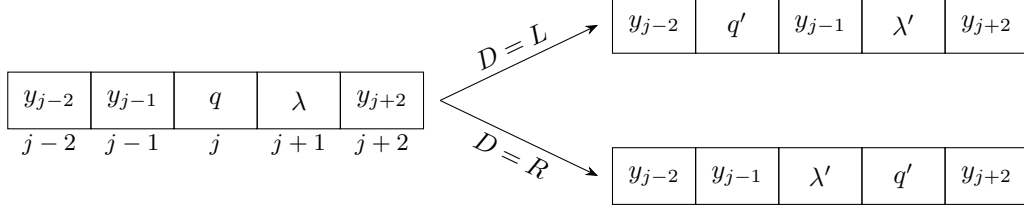


Figure 8: The tape head contents around the previous tape head’s chunk offset after a step of the Turing machine M . All indices before $j - 1$ and after $j + 1$ are identical in the new chunk. For simplicity, the indices are shown relative to the start of the chunk.

A slight wrinkle arises due to the fact that BAPOs cannot attend to the last instance of a token, which would enable the efficient “tape diff” Turing machine simulation used by Merrill and Sabharwal [24]. As such, our construction requires a fixed maximum tape size, which means different BAPO-CoTs are needed for larger problem instances—but crucially, their bandwidths are identical. This is analogous to the requirement of Merrill and Sabharwal [24] that the precision of the transformer grows with the problem instance (although in our case, the scaling increases the number of chain-of-thought steps).

Given n , we will construct a $(2, 3)$ -BAPO-CoT that simulates M on inputs of size at most n . Let $\Gamma = \Sigma \cup \Lambda \cup Q \cup \{\sqcup, \square\}$ be the token set for the BAPO-CoT. The BAPO-CoT will simulate M by writing out the contents of the tape at each step of M , along with the current state, which will be written to the left of the tape cell where the tape head is currently positioned. Since only $s(n)$ tape cells are required, the BAPO-CoT will simulate a tape with exactly $s(n)$ cells. So, on input $x_1 \dots x_n$, the first state the BAPO-CoT will write out is $q_0 x_1 \dots x_n \sqcup \dots$, with total length $c = s(n) + 1$, which we will call the chunk size. Let $\text{chunk}(i) = \lfloor i/c \rfloor$. We use m to denote the current length of the BAPO-CoT’s input (with $m = n$ at the first step) and $y = y_1, \dots, y_m$ the current BAPO-CoT input itself (with $y_1 \dots y_n = x$).

The prefix oracle f is defined as follows:

$$f(y_1 \dots y_k) = \begin{cases} 00 & \text{if the last symbol of } y_1 \dots y_k \text{ is some } q \in Q \\ 01 & \text{if the symbol to the right of the last } q \in Q \text{ in } y_1 \dots y_k \text{ is } 0 \\ 10 & \text{if the symbol to the right of the last } q \in Q \text{ in } y_1 \dots y_k \text{ is } 1 \\ 11 & \text{otherwise (} y_1 \dots y_k \text{ contains no symbols in } Q; \text{ every } q \text{ is followed by 0 or 1).} \end{cases}$$

Thanks to this f , the suffix oracle always knows the symbol to the right of the state from the previous chunk. The attention function g is defined as follows:

$$g(y_{k+1} \dots y_m, k, y_i, i) = \begin{cases} 1 & \text{if } i = n - c \\ 1 & \text{if } i = n - c - 1 \\ 1 & \text{if } \text{chunk}(i) = \text{chunk}(n) - 1 \text{ and } y_i \in Q \\ 0 & \text{otherwise} \end{cases}$$

Thanks to this g , the suffix oracle always knows (a) the symbol at the current chunk offset index in the previous chunk, (b) the symbol before the one from (a), and (c) the state in the previous chunk and its chunk offset index (i.e., the tape head position). If any of these positions are in the suffix, they are directly observed by the suffix oracle and if they are in the prefix, then they are contained in the attended set G .

The suffix oracle h performs the following procedure given $f(y_1 \dots y_k)$, G , k , and $y_{k+1} \dots y_m$:

1. If $\text{chunk}(n) = 0$, return $\sqcup$
2. If $\text{chunk}(n) = 1$:
 - (a) If $n = c + 1$: return q_0
 - (b) Else: return y_{n-c-1} (y_{n-c-1} is either in the suffix or in G)
3. Else: let $i = n \bmod c$. Let j be the head position in the previous chunk and let q be the state in the previous chunk. Note that q , j , y_{n-c} , and y_{n-c-1} are all known to h , since they

are either included in the suffix or in G (j is computable from the positional encoding of q). Moreover, $\lambda = y_{\text{chunk}(n-c) \cdot c + j + 1}$ (the symbol under the tape head in the previous chunk) is either in the suffix or can be inferred given the suffix and $f(y_1 \dots y_k)$. The suffix oracle can thus compute the step of the Turing machine $\delta(q, \lambda) = (q', \lambda', D)$, where $q' \in Q$ is the next state, λ' is the symbol written to the tape, and $D \in \{L, R\}$ is the direction the tape head moves.

First, we check to see if M has halted. If $q' = q_{\text{accept}}$: if y_m is not 1, return 1, otherwise return $\square$. If $q' = q_{\text{reject}}$: if x_m is not 0, return 0, otherwise return $\square$. This ensures we output the answer and then terminate. Otherwise, we proceed to simulate M . To output the next symbol given a step of the Turing machine M applied to the previous chunk (see Figure 8):

- (a) If $i = j - 1$: if $D = L$, return q' ; if $D = R$, return y_{n-c}
- (b) Else if $i = j$: if $D = L$, return x_{n-c-1} ; if $D = R$, return λ'
- (c) Else if $i = j + 1$: if $D = L$, return λ' ; if $D = R$, return q'
- (d) Else: return y_{n-c}

This procedure results in the updated tape of M being written out symbol-by-symbol into the next chunk. Since M decides L , it will eventually halt, at which point the above BAPO-CoT outputs the answer (0 or 1) and then outputs $\square$ to terminate. $\square$

C Experiments

C.1 Implementation Details

All models were forced to output a pre-set JSON schema, shown in the tables below. The model versions and API settings were as follows:

Family	Model	Version Specifier	Temperature	Other Params
GPT	4o	gpt-4o-2024-11-20	0	{effort: medium}
	4o mini	gpt-4o-mini-2024-07-18	0	
	o3	o3-2025-04-16	n/a	
Claude	3.5 Sonnet	claude-3-5-sonnet-20241022	0	
	3.5 Haiku	claude-3-5-haiku-20241022	0	
Gemini	1.5 Pro	gemini-1.5-pro-002	0	
	1.5 Flash	gemini-1.5-flash-002	0	
	2.5 Flash	gemini-2.5-flash-preview-04-17	0	

The experiments took ≤ 1 day and $\sim \$400$ of API credits to run (\$93 of which were for o3 alone), with preliminary experiments taking an additional $\sim \$150$ of API credits.

C.1.1 Base Experiments

Below are the instructions used for each task, illustrated using one example instance:

Experiment	Example Prompt	Example Output
------------	----------------	----------------

INDEX	Output the element at the specified index (starting at 0) of the list: List: [{"index": 0, "value": 117}, {"index": 1, "value": 30}, {"index": 2, "value": 169}, {"index": 3, "value": 113}, {"index": 4, "value": 52}, {"index": 5, "value": 168}] Index: 0	{"element_value": 117}
EQUALITY	Output true if the left and right lists are identical: Left: [1, 1, 0, 1, 0, 1, 1, 0, 0, 1] Right: [0, 1, 0, 1, 0, 1, 1, 0, 1, 1]	{"equals": false}
MATCH2	You are given a list of numbers and a number x. Determine whether $\text{list}[i] + x = 0$ for some i. List: [-300, 62, 144, -490, 469] x: -144	{"found_i": true}
REACHABILITY	You are given an directed graph with 6 nodes as a list of edges (i, j). An edge (i,j) means that node i points to j. The edges in G are: [[5, 4], [2, 0], [4, 1], [3, 2]] Is there a path from node 5 to node 1?	{"path_exists": true}
MAJORITY	Output true if the majority of elements of this list are 1, else false: [0, 1, 0, 0, 1, 1, 1]	{"majority_is_1s": true}
MATCH3	You are given a list of numbers and a number x. Determine whether $\text{list}[i] + \text{list}[j] + x = 0$ for some i, j. List: [508, 567, -178, 382, -240] x: -890	{"found_i_and_j": true}
DISJOINTNESS	These left and right lists represent sets using binary indicators for each item. Output true if these sets are disjoint and false if they have a non-empty intersection. That is, output true if and only if there is no index where both lists contain 1. Left: [0, 1, 1, 1, 0, 1] Right: [0, 0, 0, 0, 1, 0]	{"is_disjoint": true}
INTDISJOINTNESS	Output true if left and right lists are disjoint (share no elements) and false otherwise: Left: [73, 290, 133, 342, 142, 279] Right: [236, 16, 306, 144, 279, 242]	{"is_disjoint": false}
UNIQUE	Output the element in the list that occurs only once: [4, 4, 4, 4, 6]	{"unique": 6}

SETDIFF	You are given two sets of numbers A and B. Output the element in set A that is not in set B. If there is no such element, output -1. Set A: [3, 5] Set B: [5, 2, 3]	{"element": -1}
---------	---	-----------------

To generate the problem instances, we used the procedures below. We use a grid of $n \in \{6, 50, 100, 200\}$ but resulting list lengths might deviate slightly if a problem requires odd numbers. We generated an equal amount and positive and negative instances where applicable.

- INDEX: For each run, sample a permutation π of $\{0, \dots, 199\}$; set $x = \pi_{1:n-1}$, choose $i \sim \text{Unif}(0, n-1)$.
- EQUALITY: Sample $x \in \{0, 1\}^n$ uniformly. Let $y = x$ for positives. For negatives, choose $i \neq j$ with $x_i \neq x_j$, swap y_i, y_j .
- MATCH2: Generate permutation $\pi \sim \text{Perm}(0, 999)$. Set $x = \pi_{1:n}$. Then, to generate
 - Positives: inject $-x_n$ into $x_{1:n-1}$ at a random position
 - Negatives: ensure $-x_n \notin x_{1:n}$ by replacing it with a random element from π .
- REACHABILITY: Construct $k = 2$ node-disjoint paths of length $\ell = n/k - 1$. Let graph G be union of paths, map node names to integers via random bijection σ . Choose (s, t) such that:
 - Positives: s, t on same path $\Rightarrow$ path exists
 - Negatives: s, t on different paths $\Rightarrow$ no path
- MAJORITY: Generate $x \in \{0, 1\}^{n+1}$ such that majority bit occurs $\lceil (n+1)/2 \rceil$ times. Shuffle x .
- MATCH3: Sample $\pi \sim \text{Perm}(-250, 1000)$. Reject if $x = \pi_{1:n}$ has a triplet that satisfies $x_i + x_j + x_n = 0$ (negative). For positives, generate negative instance and then select $i \neq j$, set $x_n = -(x_i + x_j)$.
- DISJOINTNESS: Sample (x_i, y_i) uniformly from $\{(0, 0), (0, 1), (1, 0)\}$.
 - Positives: already disjoint
 - Negatives: pick j , set $a_j = b_j = 1$
- INTDISJOINTNESS: This is slightly different from DISJOINTNESS to avoid shortcuts from varying set sizes. Generate permutation $\pi \sim \text{Perm}(0, 399)$. Set $x = \pi_{1:n/2}$ and $y = \pi_{n/2:n}$. For negatives, set $x_j = y_j$ for random index j .
- UNIQUE: Generate permutation $\pi \sim \text{Perm}(0, n-1)$. Then, place one unique element u and fill remaining with elements of frequency ≥ 2 by drawing from $\pi_{1:n/4}$.
- SETDIFF: Generate permutation $\pi \sim \text{Perm}(0, n-1)$. Sample unique element u . Split π into parts and recombine:

$$A = S \cup \{u\}, \quad B_{pos} = S \cup V \cup \{u\}, \quad B_{neg} = S \cup V \cup \{v\}$$

where S = shared, V = unique-to-B, v = randomly sampled replacement element from S . $|S| = |V| + 1$.

C.1.2 CoT Experiments

For the chain of thought variants, we prepended the following instructions:

Think step by step on the CoT, but stay under 250 words.

The output JSON object then contained a cot field before the actual answer, e.g.,

```
{"cot": "To determine if there exist indices i and j ...", "found_i_and_j": true}
```

Additionally, we ran experiments with two reasoning models that also supported structured outputs, OpenAI's o3 and Google's Gemini Flash 2.5. Model version and parameters can be found above. Figure 9 shows the number of reasoning tokens the models used.

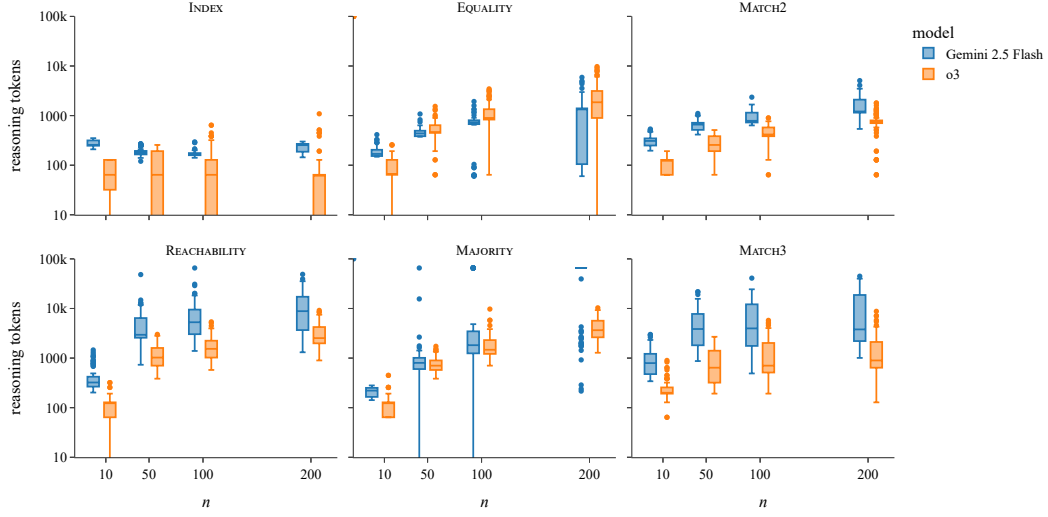


Figure 9: Number of reasoning tokens used by o3 and Gemini 2.5 Flash for each problem. The models perform well on BAPO-hard tasks in line with our BAPO-CoT result, but they use thousands or even tens of thousands of CoT tokens to do so.

C.1.3 Real-World Experiments

Experiment	Example Prompt	Example Output
VARIABLETRACKING	In the Python code below, is x7 == "a" at the end of execution? <pre> '''python x6 = "a" x4 = "b" x0 = x6 x2 = x4 x3 = x0 x8 = x2 x9 = x3 x7 = x3 x1 = x8 x5 = x8 ''' </pre>	<code>{"is_equal": true}</code>

MAJORITYREVIEW	Output true if the majority of the following reviews is positive, else false. [{ "id": 0, "review": "I loved the grand entrance hall with its two impressive chandeliers and a player grand piano. I took advantage of the exercise room in the basement and loved getting a coffee from the bar to take up to my room. The cleaning staff were particularly pleasant, greeting me every time we happened to pass. The lift [...]	{"majority_is_positive": true}
FINDNEGATIVEREVIEW	Return the id of the most negative review. [{ "id": 0, "review": "Lovely hotel and great location. I can recomend this hotel, the location is great for all tourist attractions and the airport. Very friendly staff and worth every penny." }, { "id": 1, "review": "We stayed at the Boston Park Plaza Hotel in April and couldn't have been happier. The hotel is centrally located near the Public Gardens, Theater District and Boston Commons which [...]	{"most_negative": 8}

Dataset Processing Details. We used hotel reviews from the SPACE dataset³ [2]. Reviews with a rating of 5 get a positive labels, reviews with a rating of 1 get a negative label to ensure a clear separation between classes. We annotate the resulting reviews with GPT4.1-nano to check for consistency and only keep the ones where the original label agrees with the LLM annotation. Finally, we only keep hotels with at least 101 positive and 53 negative labels to make sure we have a large enough set to subsample from.

- **VARIABLETRACKING:** Follow the same process as for REACHABILITY to construct a graph with $k = 2$ paths. Let these two paths be p and p' .
 - Then choose i, j with $j < i$ and insert a cross path edge from $p_j \rightarrow p'_i$, erasing $p'_{i-1} \rightarrow p'_i$.
 - Map nodes to variable names $x_0, x_1, \dots$ via random bijection and initialize all nodes with no incoming edges to a letter from the alphabet.
 - Choose s and t as with REACHABILITY
 - Finally, generate assignment statements via sampling a random topological ordering of the overall graph.

³Available at <https://github.com/stangelid/qt> under an MIT License.

- **MAJORITYREVIEW**: For each label $y \in \{\text{True}, \text{False}\}$, sample $n/2 + s$ reviews of label y , and $n/2$ of the opposite label, where $s = 3$ is a slack variable to help reduce any remaining noise in the reviews. Shuffle the combined list of reviews.
- **FINDNEGATIVEREVIEW**: Go through hotels in round-robin fashion. Sample $n - 1$ positive and 1 negative review to generate data for one task instance. Shuffle those in random order.

C.2 Additional Results

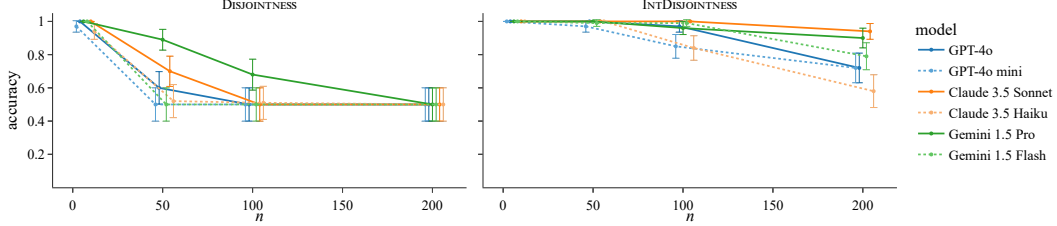


Figure 10: Additional results on BAPO-easy problems. INTDISJOINTNESS is a variant of DISJOINTNESS where sets are represented by the indices of elements they contain instead of binary vectors to show that positional encodings rather than BAPO-hardness are likely to be responsible for the poor performance on this task.

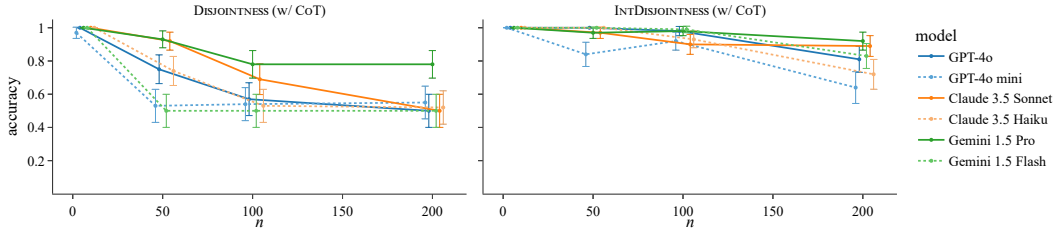


Figure 11: Adding CoT to BAPO-easy problems provides a boost to larger models on DISJOINTNESS, especially Gemini 1.5 Pro, while performance remains high on the other problems..

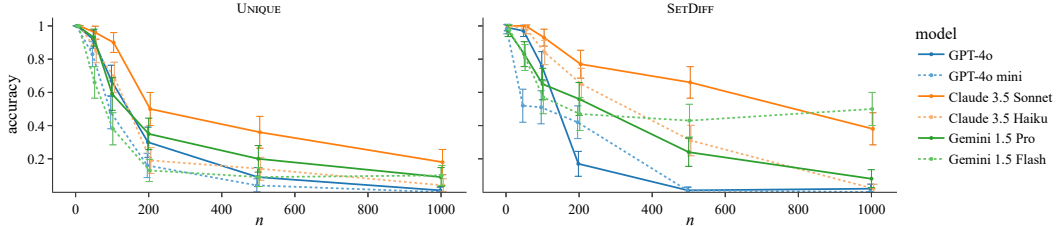


Figure 12: The hardness of UNIQUE and SETDIFF scales with the vocabulary size which we try to increase via input length n here. Drops still occur, but appear to be less pronounced, perhaps because pre-trained LLMs have fixed token representations and scaling input length is only a proxy for increased vocabulary size.

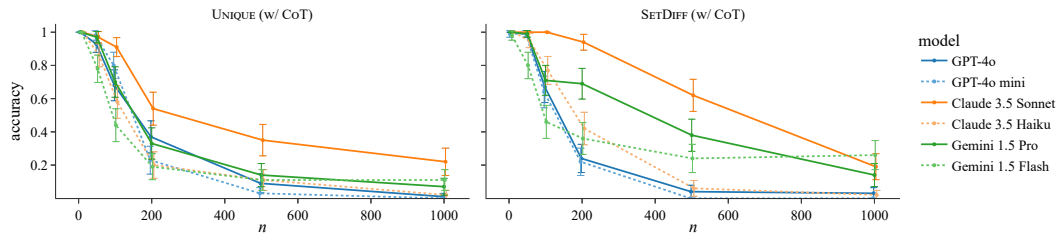


Figure 13: Adding CoT to BAPO- Σ -hard problems does not result in substantial changes. We conjecture that this is again due to the static nature of the underlying vocabulary representations.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [\[Yes\]](#)

Justification: We clearly state in the abstract and introduction what are hypotheses are, what our theoretical results state, and what our experiments indicate.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Section 6 discusses the limitations of our work.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[Yes\]](#)

Justification: All results are stated precisely and have proofs in Section B.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: Section C.1 includes all experimental details, including exact models, prompts, and data generation procedures.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All of our code is available at <https://github.com/microsoft/bapo>. The code includes instructions for downloading public data and for running the experiments.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [Yes]

Justification: All details are available in Section C.1.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: As described in Section 4, all of our plots show 95% *t*-test confidence intervals across $n = 100$ runs.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer “Yes” if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section C.1 states that the experiments are reproducible with in less than a day with ~\$400 of API credits, with preliminary experiments taking an additional ~\$150 of API credits.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We see no potential for societal harm or other ethical concerns in this research.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [NA]

Justification: As our work proposes and analyzes a mathematical model of LLM capability, we see no broader societal impact.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: Our paper does not provide data or models (in the sense of an LLM; we do introduce a mathematical model).

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We cite authors of the SPACE dataset, link to the data source, and explicitly mention the MIT License under which it is released in Section C.1.3.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.

- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: We attach our code along with documentation as Supplementary Material.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[NA\]](#)

Justification: The paper does not concern research with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [\[NA\]](#)

Justification: The paper does not concern research with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.

- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

Answer: [NA]

Justification: LLMs are our object of study, but they are not a part of our research methodology.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (<https://neurips.cc/Conferences/2025/LLM>) for what should or should not be described.